WTR 最终考核

从零配置到电机闭环
分步骤实作教程

STM32F407VET6 + FreeRTOS + USART DMA + CAN + 串级 PID

适合刚完成 STM32 和 FreeRTOS 入门练习的同学。每一步先说明要解决什么问题，再说明怎么操作、数据怎么走，以及如何判断这一小块已经完成。

这份文档包含	使用方式
18 个实作步骤 + 2 个附录	按顺序完成；每步验收通过，再接入下一模块。
每步逻辑图、目的、函数与变量说明	先看图理解整体关系，再看对应代码。
CubeMX 参数、文件划分、核心算法代码	代码按文件说明放置；明确标为伪代码的部分需要按本步规则实现。
错误排查、联调、视频与提交检查表	完成通信测试后才开启电流输出。

依据：用户提供的《WTR最终考核》、随附 CRC 与电机/电调手册。原始资料目录：D:\WTR\WTR_homework\大作业\大作业。

版本：2026-10-20。本文是教学实施方案，不是已经在你的实物上验证通过的工程，也不是可以直接作为“完成实验”的报告。接线、上位机协议细节和 PID 参数仍需现场验证。

阅读路线与页码

第一次阅读先看步骤 01；第一次动手做到步骤 04 即可。不要一次把所有模块复制进去。后续对话可以直接说“开始步骤 05”，再逐块实现。

01 把考核要求变成可验证的模块 3

02 确认板子、接线与方向 5

03 在 CubeMX 中建立基础工程 7

04 建立任务、队列与文件结构 9

05 先把 8 字节协议算对 12

06 启动串口循环 DMA，把字节交给任务 14

07 在 RxTask 中处理拆包、粘包和坏帧 17

08 DMA 发送二进制反馈 19

09 加入连接判断、LED 与断线急停 21

10 接通 CAN，只接收反馈 23

11 计算输出轴速度与多圈位置 25

12 用小电流验证方向和限幅 27

13 实现位置式 PID 速度环 29

14 用位置外环和速度内环构成串级 31

15 把串口命令接入唯一的控制任务 33

16 以状态机执行两组预设动作 35

17 选做：给位置目标加入梯形速度规划 38

18 联调、排错并准备提交 40

19 附录 A：协议测试帧与算法自检 43

20 附录 B：C 语法、接口边界与资料索引 45

01 把考核要求变成可验证的模块

本步目的：明确最终必须交什么、哪些是硬性要求、哪些是本教程的设计选择。



先建立正确的整体认识

之前作业是随机传感器数据、队列和串口输出。这次是真实电机控制：上位机告诉你目标，主控读取电机真实反馈，通过 PID 算出电流命令，再由电调驱动电机。FreeRTOS 负责安排这些工作，不负责自动把电机调好。

考核内容	实际要完成的事情	对应步骤
任务一：通信	USART1 接收与发送稳定阶段均用 DMA；坏帧丢弃；任务内解析；断线急停；LED0 表示连接。	05-09
任务二：电机	CAN 反馈、输出轴转速与多圈位置、电流控制、位置式速度 PID、位置外环串级控制。	10-14
任务三：综合	串口切换速度/位置；执行两组固定动作；反馈状态与速度位置。	15-16
选做	梯形速度规划后进行位置闭环。	17
提交	全部工程代码、.ioc、成功演示视频。	18

本文采用的实现方案

任务分工：InitTask 只做一次启动；RxTask 接收和解析串口字节；ControlTask 每 1 ms 更新电机状态和控制；TxTask 周期性发送状态。LED 在 ControlTask 中随连接状态更新，不必为了一个 LED 再建任务。本题不要求按键，也不需要用软件定时器生成随机数。

项目	已知要求 / 教程选择
实际主控	按考核原理图使用 STM32F407VET6。CoreV2.1.pdf 出现 FDCAN、UART9/10 等另一版资源，不能混用其引脚。若现场发的确实是 V2.1，先重新核实型号。
通信初值	教程选择 115200、8N1，系统 tick=1000 Hz，状态反馈 50 Hz。题目未固定这些值。
CRC 范围	参数已给定；覆盖哪些字节未明说。本文按前 7 字节（含帧头，不含 CRC）计算，须用上位机实际帧确认。
速度模式 A 的单位	题目只写最大电流/力矩，未写缩放。本文内部统一用 CAN 原始电流计数；若上位机传 mA，必须先换算。

项目	已知要求 / 教程选择
断线与到位	本文先选主机超时 300 ms、CAN 超时 50 ms、误差阈值和保持时间，均是待实测调整的设计参数。
自动重连	通信可以自动恢复；断线后先保持零电流，收到零速确认后再接受运动命令，避免突然恢复旧目标。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
本步不新增函数	先形成模块边界和验收清单，避免一开始把所有代码塞进一个任务。

变量说明：名称含义与保存内容

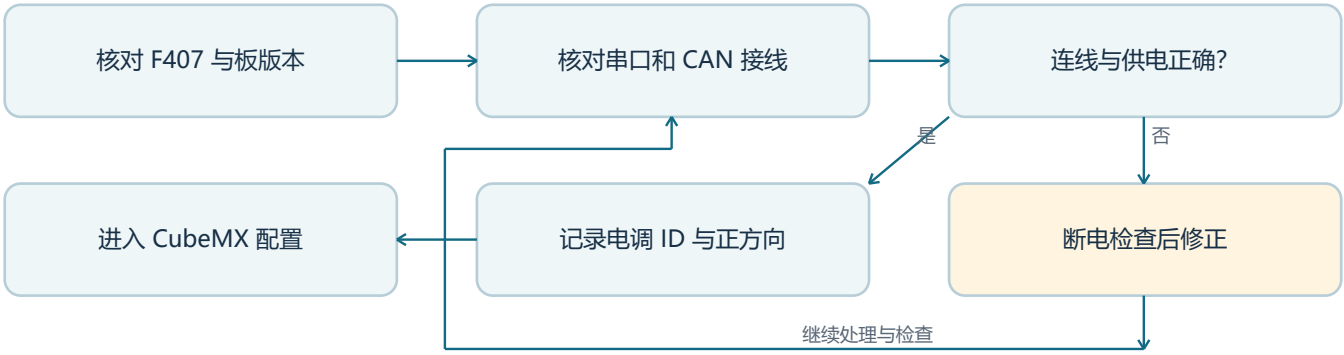
名称与类型	含义、用途与所属模块
模式、目标、反馈、故障	分别表示“做什么”“希望达到什么”“实际是多少”“能否继续运行”；后面分别放进命令、反馈和状态结构体。

本步验收：通过后再继续

你能说清“串口收到目标 → 任务解析 → PID → CAN 电流 → 电机反馈 → 状态上传”这一闭环，并知道断线时要直接停输出，而不是只把 LED 熄灭。

02 确认板子、接线与方向

本步目的：确保后续软件配置对应真实硬件，先把电机固定并保持零电流。



以考核 PDF 第 2 页的原理图为准

用途	本次引脚 / 参数	你要做的操作
USART1 TX	PB6，AF7	主控发送；连接串口模块 RX。
USART1 RX	PB7，AF7	主控接收；连接串口模块 TX。
CAN1 RX / TX	PA11 / PA12，AF9	使用板载 CAN 收发器和 CANH/CANL 接口，不能用 GPIO 直接连 CAN 总线。
LED0	PE7，高电平亮	在 CubeMX 标记为 LED0；默认输出低。
外部晶振	8 MHz，PH0 / PH1	RCC 选择 Crystal/Ceramic Resonator。
调试	PA13 SWDIO / PA14 SWCLK	SYS 选择 Serial Wire；接调试器和地。
电机与电调	M3508+C620 或 M2006+C610	一次先接一台，设置电调 ID=1，反馈 ID=0x201。

旧作业里的 PA9/PA10、PA5 不能直接沿用。MX 接口具体线序应与板上丝印或接线资料核对；不要仅按插头外形猜线。串口采用 3.3 V TTL 电平，TX/RX 交叉，共地。

现场上电顺序

① 断电连接主控、CAN 分电板、电调和电机，固定电机本体，输出轴保持空载且周围没有易卷入物。② 核对电源极性与额定电压。③ CAN 两端终端电阻通常各 120 Ω，断电测 CANH-CANL 约 60 Ω；已有终端时不要重复堆叠。④ 先只运行反馈代码，确认没有发非零电流。⑤ 最后才进入小电流测试。

考核提到施加阻力观察速度跟随。应使用固定、可控的加载装置或在指导人员监督下验证，避免手抓旋转轴或装有附件的转子。零电流会使轴自由滑行，它不是机械刹车；有重力负载时还需要独立机械保护。

变量说明：名称含义与保存内容

名称与类型	含义、用途与所属模块
MOTOR_ID = 1	教程选择的电调编号；对应反馈 0x201、控制帧 0x200 的第 1 组两字节。
MOTOR_DIR = +1 或 -1	方向校准系数；输出轴顺时针为正。反馈和电流命令必须使用同一个系数。
gear_ratio	减速比：M2006=36；M3508 使用 3591/187，约 19.2032。

函数说明：谁提供，谁调用，为什么需要

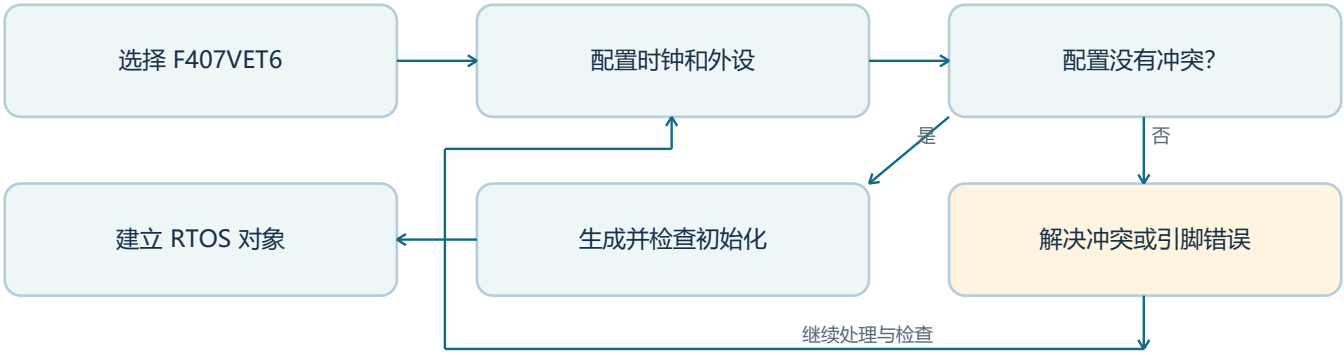
名称 / 来源	目的、参数与结果
HAL_GPIO_WritePin / HAL 库	写 LED 引脚，三个参数为端口、引脚、SET/RESET；在连接判断完成后调用。

本步验收：通过后再继续

照片记录接线和板型号；确认 LED0 是 PE7；知道本次用哪种电机/电调和 ID。未确认板版本时可以继续阅读，暂不按错误引脚上电。

03 在 CubeMX 中建立基础工程

本步目的：一次配置好时钟、外设、DMA 与中断，使后续模块拥有可靠基础。



新建工程与系统时钟

打开 STM32CubeMX → New Project → MCU Selector → STM32F407VET6。创建独立目录，例如 D:\WTR\WTR_homework\final_project。Project Manager 选择工具链 CMake（若当前版本提供），工程名称 final_project；打开 Keep User Code when re-generating。

System Core → SYS：Debug=Serial Wire，Timebase Source=TIM6。RCC：HSE=Crystal/Ceramic Resonator。Clock Configuration：HSE=8 MHz，PLL Source=HSE，PLLM=8，PLLN=336，PLLP=2，PLLQ=7；SYSCLK=168 MHz、AHB=168 MHz、APB1=42 MHz、APB2=84 MHz。HAL 的毫秒时间由 TIM6 维护，RTOS 的节拍使用 SysTick，二者分工明确。

USART1 与 DMA

CubeMX 位置	填写内容
Connectivity → USART1	Asynchronous；PB6 TX、PB7 RX；115200；8 bit、None parity、1 stop、无硬件流控。
USART1 → DMA Settings → RX	Peripheral to Memory；Circular；Byte/Byte；Peripheral Increment 关闭；Memory Increment 开启。可用 DMA2 Stream2 Channel4，以 CubeMX 合法分配为准。
USART1 → DMA Settings → TX	Memory to Peripheral；Normal；Byte/Byte；Memory Increment 开启。常用 DMA2 Stream7 Channel4。
NVIC	USART1、RX DMA、TX DMA 中断都开启。建议这些中断抢占优先级统一为 6，子优先级 0。
内存	RX/TX 缓冲区放普通 SRAM，不要放 F407 的 CCM（0x10000000 区域）；DMA 无法访问 CCM。

DMA 模式中 Normal 表示一次传完停止，Circular 表示写到末尾后从开头继续。接收用 Circular 防止每包重启的空窗；发送用 Normal，让每次完成回调表示一帧确实发完。不要因为“用了 DMA”就关闭 USART1 中断，IDLE 接收事件和普通 DMA 发送最后完成仍需要它。

CAN1 与 LED

配置	内容与理由
CAN1	Activated，Normal mode；PA11 RX、PA12 TX；APB1=42 MHz。
位时序	Prescaler=3；BS1=11 TQ；BS2=2 TQ；SJW=1 TQ。42 MHz ÷ [3×(1+11+2)] = 1 Mbps。
功能选项	Time Triggered 关闭；Auto Wakeup 关闭；Auto Bus-Off 开启；Auto Retransmission 开启。运行时仍须检查 bus-off 和发送拥塞。
CAN 中断	RX FIFO0 中断开启，抢占优先级 6；状态错误中断按错误处理启用。后续代码还要调用 ActivateNotification。
GPIO PE7	Output Push Pull、No pull、Low speed；用户标签 LED0；默认 Low。

数值越小的 NVIC 抢占优先级越高。假设 FreeRTOS 的 configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY=5，则调用 RTOS API 的中断可设为 5～15，不能设为 0～4。这里选 6；不要把线程的 High/Normal 与 NVIC 的数字优先级混为一谈。USART 和 RX DMA 用同一抢占优先级，还能防止它们嵌套修改接收游标。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
MX_GPIO_Init / MX_DMA_Init / MX_USART1_UART_Init / MX_CAN1_Init	CubeMX 生成的初始化函数，main 中调用；负责外设寄存器和 DMA 关联，但不会自动完成应用层的接收启动或电机协议。
SystemClock_Config	CubeMX 生成，配置时钟；通常不必手写修改。
HAL_Init / HAL 库	初始化 HAL 基础；不是启动 FreeRTOS 的函数。

变量说明：名称含义与保存内容

名称与类型	含义、用途与所属模块
huart1 / UART_HandleTypeDef	CubeMX 定义的串口句柄，库通过它知道操作 USART1 和关联 DMA。
hcan1 / CAN_HandleTypeDef	CAN1 句柄。
LED0_GPIO_Port / LED0_Pin	CubeMX 在 main.h 生成的宏，分别代表 GPIOE 和 GPIO_PIN_7。

本步验收：通过后再继续

CubeMX 无红色引脚冲突；生成的 usart.c 确实是 PB6/PB7；can.c 的位时序算出 1 Mbps；TIM6 与 SysTick 分工正确。

04 建立任务、队列与文件结构

本步目的：先把“谁负责什么、通过什么传递数据”固定下来，防止重复创建和共享数据混乱。



FreeRTOS 配置

Middleware → FREERTOS → Interface 选择 CMSIS_V2。开启抢占调度 configUSE_PREEMPTION=1；tick rate=1000 Hz；动态分配开启，选择 heap_4；TOTAL_HEAP_SIZE 可先给 32768 字节。开启 malloc failed hook 与栈溢出检查时，记得提供对应钩子实现。以上是 F407 本工程的起点，最终根据内存占用测量。

Task Name	Entry Function	Priority	栈：Words / 字节	Code Generation
initTask	InitTask	High	256 / 1024	As external
controlTask	ControlTask	High	512 / 2048	As external
rxTask	RxTask	AboveNormal	512 / 2048	As external
txTask	TxTask	Normal	384 / 1536	As external

把 defaultTask 改为 initTask 即可，不需要保留一个空闲 defaultTask。Allocation 选 Dynamic，Parameter 填 NULL。Words 是 4 字节一项；CMSIS osThreadAttr_t.stack_size 则是字节，生成代码通常会写 512*4。核对生成代码而不是凭 GUI 记忆。High 只保证优先响应，控制任务每轮仍需阻塞等待下个周期，否则会饿死低优先级任务。

先创建 App/Inc/app_types.h，再设置队列元素类型

可直接新建的公共类型头文件；结构体用于任务间复制，不等于串口线上格式。

```
#ifndef APP_TYPES_H
#define APP_TYPES_H
#include <stdint.h>

typedef struct {
    uint32_t tick;
    uint16_t len;
    uint8_t data[256];
} RxBlock;

typedef struct {
    uint32_t id;
    uint32_t tick;
    uint8_t data[8];
} CanFrame;

typedef struct {
    uint8_t mode;
    int16_t a;
```

```
    int16_t b;
    uint32_t action_id;
    uint32_t tick;
} Command;

typedef struct {
    uint8_t state;
    float speed_rpm;
    float position_deg;
} Telemetry;
#endif
```

队列名	长度	Item Size / 元素类型	用途
rxBlockQueue	8	RxBlock / sizeof(RxBlock)	中断复制串口新增字节；RxTask 取走。
canRxQueue	64	CanFrame / sizeof(CanFrame)	保留连续 CAN 反馈，ControlTask 顺序展开多圈角度。
commandQueue	16	Command / sizeof(Command)	RxTask 把通过检查的命令交给控制任务。
telemetryQueue	8	Telemetry / sizeof(Telemetry)	控制任务每 20 ms 提交状态快照；TxTask 取走。

CubeMX 队列界面若填写的是类型，填 RxBlock 等类型名；若是字节表达式，则填 sizeof(RxBlock)。生成后最终必须是 osMessageQueueNew(8, sizeof(RxBlock), ...)，不能只填 4 或填结构体指针的大小。freertos.c 的 USER CODE Includes 区加入 app_types.h，使类型在创建队列前可见。

新建文件夹与文件

VS Code 打开工程根目录，即含 CMakeLists.txt、Core、Drivers 的那层。资源管理器右键根目录 → New Folder → App，再建 Inc 和 Src。后续每步再增加对应模块，不要一次创建带未知函数调用的所有文件。

文件	职责
app_types.h / app_config.h	公共消息类型 / 配置常量，不放任务无限循环。
rtos_handles.h	只声明 CubeMX 已创建对象的 extern 句柄。
init_task.c/.h	唯一的启动流程。
protocol.c/.h	CRC、8 字节协议解码和编码、流式组帧。
uart_rx.c/.h + rx_task.c/.h	DMA 字节搬运 / 任务内解析与恢复。
uart_tx.c/.h + tx_task.c/.h	DMA 发送完成通知 / 周期状态发送。
motor_can.c/.h + motor_feedback.c/.h	CAN 硬件接口 / 角度和转速换算。
pid.c/.h + preset.c/.h	可复用 PID / 两组动作状态机。
control_task.c/.h	唯一持有目标、反馈、PID 和当前动作状态的模块。

rtos_handles.h：核对 freertos.c 的真实名字；extern 是声明，不要在此再次创建队列。

```
#ifndef RTOS_HANDLES_H
#define RTOS_HANDLES_H
#include "cmsis_os2.h"
extern osMessageQueueId_t rxBlockQueueHandle;
extern osMessageQueueId_t canRxQueueHandle;
extern osMessageQueueId_t commandQueueHandle;
extern osMessageQueueId_t telemetryQueueHandle;
extern osThreadId_t rxTaskHandle;
extern osThreadId_t txTaskHandle;
```

```
extern osThreadId_t controlTaskHandle;
#endif
```

让 CMake 找到自建文件

根 CMakeLists.txt 中添加；以后每新增实际参与编译的 .c，在此追加一行。头文件不必逐个加入 sources。

```
# Put this after add_executable(${CMAKE_PROJECT_NAME}).
target_include_directories(${CMAKE_PROJECT_NAME} PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/App/Inc
)
target_sources(${CMAKE_PROJECT_NAME} PRIVATE
    App/Src/init_task.c
    App/Src/control_task.c
    App/Src/rx_task.c
    App/Src/tx_task.c
)
```

如果希望后续新 .c 自动进入构建，可以改用 file(GLOB APP_SOURCES CONFIGURE_DEPENDS "${CMAKE_CURRENT_SOURCE_DIR}/App/Src/*.c")，再 target_sources(... PRIVATE ${APP_SOURCES})。二选一即可；显式列出更容易看清编译了什么。文件夹路径改变后仍要重新配置，不是 GLOB 能解决所有缓存问题。

当前先给四个任务各写一个可编译空壳（包含 cmsis_os2.h 和对应头文件）。InitTask 此时只 osThreadExit()；后面再补启动。

```
void ControlTask(void *argument)
{
    (void)argument;
    for (;;) {
        osDelay(1U);
    }
}
```

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
osMessageQueueNew / CMSIS 库	创建能保存固定大小消息的队列，由 CubeMX 在 MX_FREERTOS_Init 中调用；返回句柄，NULL 表示失败。
osThreadNew / CMSIS 库	创建任务并绑定入口；已在 CubeMX 配置的任务不要手动再次创建。
osThreadExit / CMSIS 库	结束当前 InitTask，释放其动态线程资源；初始化动作不会因此撤销。
osDelay / CMSIS 库	让当前任务阻塞指定 tick，让其他任务有机会执行；这里 1 tick=1 ms。

变量说明：名称含义与保存内容

名称与类型	含义、用途与所属模块
xxxHandle	handle 意为句柄；保存系统对象的标识，其他模块用它访问相同对象。
argument / void *	统一任务入口参数；没有传数据就用 (void)argument 表示暂不使用。
TOTAL_HEAP_SIZE	RTOS 动态内存总池；不同于单任务栈，也不同于链接脚本的 C 堆。

本步验收：通过后再继续

四个任务和四个队列各创建一次；每个句柄非 NULL；空工程可编译、下载、运行。若 As external 入口 unresolved，检查 .c 是否加入 target_sources、函数名是否大小写完全一致。

05 先把 8 字节协议算对

本步目的：不依赖 DMA 和电机，先验证长度、端序、CRC 与有符号数，避免把协议错误误认为硬件故障。



偏移	主控接收	主控发送
0、1	AA BB	BB AA
2	00 速度 / 11 位置 / 22 动作	状态 S
3、4	A：小端 int16	转速：小端 int16，rpm
5、6	B：小端 int16	位置：小端 int16，deg
7	CRC8	CRC8

小端表示低字节先发。例如 200=0x00C8，发 C8 00；-200 的 16 位补码是 FF38，发 38 FF。CAN 电调协议恰好相反，稍后按高字节在前处理。不要把整个 C 结构体直接发送：编译器可能补齐对齐字节，字段排列也不能代替显式协议。

新建 protocol.h 与 protocol.c

protocol.c：包含 <stdbool.h>、<stdint.h> 和 app_types.h；头文件声明两个 Protocol_ 公共函数，ReadI16LE 为本文件私有。

```
uint8_t Protocol_Crc8(const uint8_t *data, uint32_t len)
{
    uint8_t crc = 0U;
    for (uint32_t i = 0U; i < len; ++i) {
        crc ^= data[i];
        for (uint8_t bit = 0U; bit < 8U; ++bit) {
            crc = (crc & 0x80U)
                ? (uint8_t)((crc << 1) ^ 0x07U)
                : (uint8_t)(crc << 1);
        }
    }
    return crc;
}

static int16_t ReadI16LE(const uint8_t *p)
{
    uint16_t u = (uint16_t)p[0] | ((uint16_t)p[1] << 8);
    int32_t value = (u < 0x8000U) ? (int32_t)u
                                  : (int32_t)u - 65536;
    return (int16_t)value;
}

bool Protocol_Decode(const uint8_t frame[8], Command *out)
{
    if (frame[0] != 0xAAU || frame[1] != 0xBBU) return false;
    if (Protocol_Crc8(frame, 7U) != frame[7]) return false;
```

```
    if (frame[2] != 0x00U && frame[2] != 0x11U &&
        frame[2] != 0x22U) return false;
    out->mode = frame[2];
    out->a = ReadI16LE(&frame[3]);
    out->b = ReadI16LE(&frame[5]);
    out->action_id = (uint32_t)frame[3]
                   | ((uint32_t)frame[4] << 8)
                   | ((uint32_t)frame[5] << 16)
                   | ((uint32_t)frame[6] << 24);
    out->tick = 0U;
    return true;
}
```

这里把附件 C++ 模板库的同一 CRC 参数用普通 C 实现，便于你理解，不需要把所有 .c 改成 .cpp。独立校验串 "123456789" 应算出 F4。这只能验证 CRC 算法；前 7 字节是否就是上位机的校验范围，还要抓取上位机完整帧核对。

解码通过后还要做数值检查

速度/位置模式的 A 是“最大值”，负值应判为无效；最大值为 0 应明确表示禁止输出，不要解释成无限制。速度模式电流上限还要夹到本电调硬上限；位置模式速度上限夹到你验证过的工作范围。动作模式只接受两个已定义 ID。CRC 正确不等于命令在业务上有效。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
Protocol_Crc8 / 自建	输入字节地址与长度，返回一个校验字节；接收检查和发送打包共同使用。
ReadI16LE / 自建 static	把两个小端字节还原为有符号 16 位数；先组合无符号数再进行有符号范围转换。
Protocol_Decode / 自建	检查完整帧并写入 out 指向的结构体；返回 true/false。不在内部启动电机。

变量说明：名称含义与保存内容

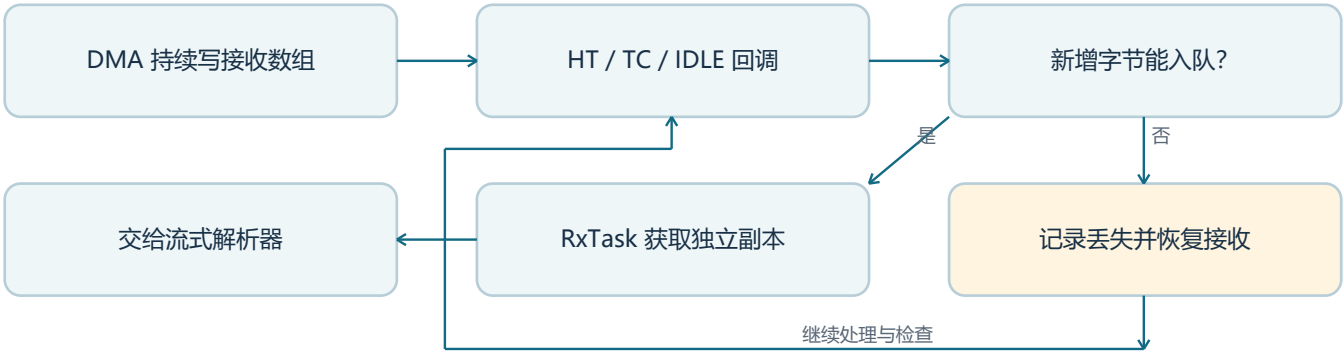
名称与类型	含义、用途与所属模块
frame / uint8_t[8]	线上收到的原始 8 个字节。
out / Command *	指向调用者准备的结构体，函数通过 out->字段 写入结果。
crc / uint8_t	当前校验余数，初始为 0，每次移位只保留低 8 位。
a、b、action_id	同一组 4 字节按模式解释；动作 ID 不是 a+b，而是四字节拼成的 uint32_t。

本步验收：通过后再继续

正负速度样例都能正确解码；改动任意一个数据字节但不更新 CRC 时必须拒绝；不支持的模式和动作 ID 不得驱动电机。步骤 19 附录提供完整测试帧。

06 启动串口循环 DMA，把字节交给任务

本步目的：中断只搬运新增字节，解析留在 RxTask，满足考核的 task 处理要求。



先理解：DMA 收到一批，不等于收到一帧

DMA 是外设把字节自动写进内存的硬件搬运器。256 字节接收数组是循环缓冲区。半满 HT、全满 TC、串口空闲 IDLE 都可能触发回调；这三个事件只是提醒“有新数据”，不是协议的分隔符。一帧可能分两次到达，一次回调也可能包含多帧。

HAL_UARTEx_RxEventCallback 是 HAL 规定名字的回调：函数体由你写，外设事件发生后由 HAL 在中断中调用。Size 表示目前写到的边界位置，不一定是本次新增字节数。不要只处理 Size==8，也不要每次从 rx_dma[0] 开始解析。

uart_rx.c 中的接收骨架

包含 usart.h、app_types.h、rtos_handles.h。此骨架要求 USART1 与 RX DMA 中断同一抢占优先级，保留 HT 中断，并按下文实现错误恢复。

```
static uint8_t rx_dma[256];
static uint16_t old_pos;
static RxBlock rx_copy;
volatile uint32_t rx_fault;

HAL_StatusTypeDef UartRx_Start(void)
{
    old_pos = 0U;
    return HAL_UARTEx_ReceiveToIdle_DMA(
        &huart1, rx_dma, sizeof(rx_dma));
}

void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart,
                                uint16_t size)
{
    if (huart->Instance != USART1) return;
    if (size == 0U || size > sizeof(rx_dma)) return;
    if (size == old_pos) return;

    uint16_t count = (size > old_pos) ? size - old_pos
                          : sizeof(rx_dma) - old_pos + size;
    rx_copy.len = count;
    rx_copy.tick = osKernelGetTickCount();
    for (uint16_t i = 0U; i < count; ++i) {
        rx_copy.data[i] = rx_dma[(old_pos + i) % 256U];
    }
    old_pos = size;
    if (osMessageQueuePut(rxBlockQueueHandle,
                        &rx_copy, 0U, 0U) != osOK) {
        rx_fault = 1U;
    }
```

```
}
```

例子：第一次 Size=5，复制 0～4；下一次 Size=8，复制 5～7；到 256 后下一次 Size=6，则复制缓冲区开头 0～5。保留 old_pos=256 可以识别重复边界事件；不要把每次回调当成全新的包。HAL 版本若没有 ReceiveToIdle_DMA，先确认是否包含对应声明和实现，再更新兼容的 STM32CubeF4 固件包，不能只手抄函数声明。

为什么需要复制？DMA 可能马上覆盖原数组。队列放 RxBlock 整体内容，才能让任务慢一点处理时仍拿到稳定数据。不要只把 rx_dma 指针入队。115200、8N1 每字节约 86.8 μs，半个 256 字节缓冲区约 11.1 ms；中断延迟与搬运必须远小于这个时间。RTOS 队列只能吸收短时突发，不能弥补长期处理速度不足。

InitTask 负责启动，其他任务等启动完成

各任务开始先 osThreadFlagsWait(APP_READY, osFlagsWaitAll, osWaitForever)。APP_READY 应放公共 app_config.h。

```
#define APP_READY (1U << 0)

void InitTask(void *argument)
{
    (void)argument;
    if (UartRx_Start() != HAL_OK) Error_Handler();
    /* Add MotorCan_Start() here after step 10 passes. */
    osThreadFlagsSet(rxTaskHandle, APP_READY);
    osThreadFlagsSet(txTaskHandle, APP_READY);
    osThreadFlagsSet(controlTaskHandle, APP_READY);
    osThreadExit();
}
```

InitTask 运行时，内核已经启动，CubeMX 创建的队列也已经存在，因此中断可以安全入队。不要在 main 的外设初始化之后、队列创建之前就启动接收回调。Error_Handler 通常关中断并死循环，适合此处初始化失败；电机运行后的故障不能只调用它，必须先执行停机策略。

必须补齐的恢复流程

HAL_UART_ErrorCallback 只设置 rx_fault，不在中断中打印或反复重启。RxTask 每轮、以及等待队列超时后检查 rx_fault。发生溢出/错误时：先请求控制层急停，停止接收相关中断与 DMA，调用 HAL_UART_AbortReceive；清空 rxBlockQueue，重置解析器和 old_pos；清除相应错误状态，重启 ReceiveToIdle_DMA，成功后恢复相关中断。失败则隔 10 ms 重试。不要调用会同时中断 TX 的全串口 Abort 代替 AbortReceive。

接收游标、队列清空与重启必须形成一个受保护的恢复过程；处理中断标志时用短临界区，避免“中断刚置 1，任务又清成 0”的竞态。不得用 volatile 代替队列或完整同步。恢复后必须重新收到完整有效帧才能认为连接恢复。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
UartRx_Start / 自建	启动一次循环接收，返回 HAL 状态；正常期间不每帧重启。
HAL_UARTEx_ReceiveToIdle_DMA / HAL 库	传入句柄、接收数组、容量，启动 DMA 与 IDLE 检测；立即返回，不等待整帧。
HAL_UARTEx_RxEventCallback / 用户实现的 HAL 回调	由 HAL 在 ISR 调用，复制增量并入队；不是普通任务，也不是软件定时器回调。
osMessageQueuePut / CMSIS 库	队列复制一个 RxBlock；参数分别为队列、消息地址、消息优先级、等待 tick。ISR 中 timeout 必须为 0。
HAL_UART_AbortReceive / HAL 库	停止本次接收，供任务中的恢复流程使用；与发送中止分开。
osThreadFlagsSet / CMSIS 库	给指定任务置启动位；不是给所有任务自动广播。

变量说明：名称含义与保存内容

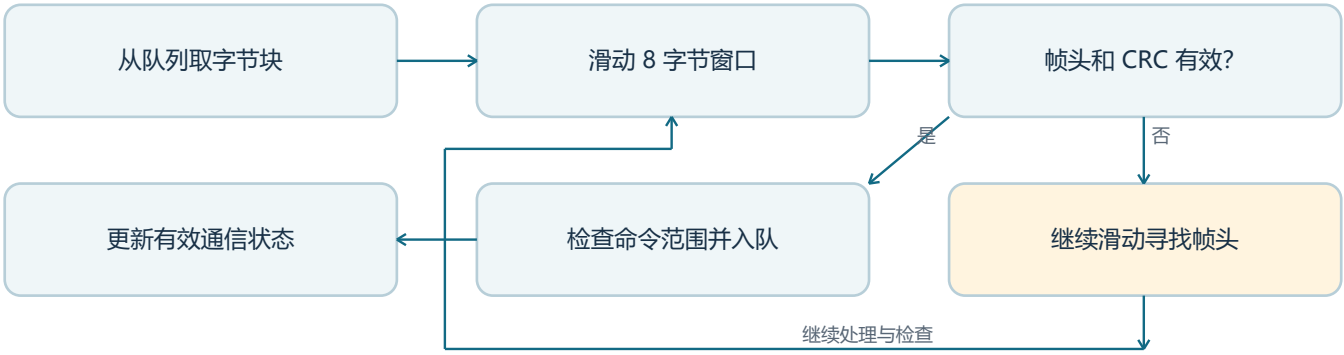
名称与类型	含义、用途与所属模块
rx_dma / uint8_t[256]	DMA 实际写入的普通 SRAM 数组，生命周期贯穿运行。
old_pos / uint16_t	上次已处理到的位置，属于接收模块。
rx_copy / RxBlock	ISR 搬运用的暂存副本；两个接收中断不能嵌套访问它。
rx_fault / volatile uint32_t	接收故障提示，不是调试计数。置位、读取和清除需要约定同步规则。

本步验收：通过后再继续

仅发送任意字节，观察 rx_copy.len 和队列变化；串口连发超过 256 字节也能连续接收。不接电机输出。连续流不能只靠 IDLE，因此不要关掉 HT/TC。

07 在 RxTask 中处理拆包、粘包和坏帧

本步目的：无论数据怎样分批到达，都能恢复出完整命令，并且不让错误帧刷新连接状态。



为什么不能直接强转结构体

例如本次拿到 AA BB 00，下一次拿到剩下 5 字节，必须拼起来才有一帧。再如一批 16 字节可能是两帧。最容易理解的办法是保留最近 8 字节，填满后每收到一个新字节，就检查一次；合法帧用完后清空窗口。损坏帧不整批丢弃，而是继续逐字节滑动。

放 protocol.c，头文件公开 Parser 类型和两个函数；memmove 来自 <string.h>，允许源与目的内存重叠。

```
typedef struct {
    uint8_t bytes[8];
    uint8_t used;
} Parser;

void Parser_Reset(Parser *p)
{
    p->used = 0U;
}

bool Parser_Push(Parser *p, uint8_t byte, Command *out)
{
    if (p->used == 8U) {
        memmove(&p->bytes[0], &p->bytes[1], 7U);
        p->used = 7U;
    }
    p->bytes[p->used++] = byte;
    if (p->used != 8U) return false;
    if (!Protocol_Decode(p->bytes, out)) return false;
    p->used = 0U;
    return true;
}
```

RxTask 的实施顺序

① 启动时 Parser_Reset。② osMessageQueueGet 等待 RxBlock，超时 10 tick，以便无人发送时仍检查恢复标志。③ 得到块后逐字节调用 Parser_Push。④ 解析成功后验证模式、A 非负和 action_id；把完成这帧所在 block.tick 赋给 cmd.tick。⑤ 用 timeout=0 把 Command 放进 commandQueue。⑥ 入队失败意味着控制侧未及时处理，置通信故障并停止接受旧运动命令，不能仅忽略返回值。

流程片段：Command_Validate 与 Comm_RequestFault 是本教程要求自建的接口，规则分别见步骤 05 和 09，不是 HAL 函数；实现前不能直接粘贴编译。

```
/* Core loop fragment inside RxTask, after a block is received. */
for (uint16_t i = 0U; i < block.len; ++i) {
    Command cmd;
    if (!Parser_Push(&parser, block.data[i], &cmd)) continue;
```

```
    if (!Command_Validate(&cmd)) continue;
    cmd.tick = block.tick;
    if (osMessageQueuePut(commandQueueHandle,
                          &cmd, 0U, 0U) != osOK) {
        Comm_RequestFault();
        break;
    }
}
```

控制任务取出命令时还要检查其 tick，过期队列消息不能当成“刚刚收到”。正常情况下控制任务每毫秒清空命令队列，处理每条命令；同时对过载设置上限，避免异常输入使控制周期无限延长。不要让 task 一次拿一个字节后就 osDelay(1)，那会把吞吐量限制得很低。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
Parser_Reset / 自建	清空半帧状态；上电和接收故障恢复时使用。
Parser_Push / 自建	喂一个新字节；成功组出一帧时返回 true 并填写 out。
memmove / C 标准库	搬移可能重叠的 7 个字节；memcpy 对重叠内存没有保证。
osMessageQueueGet / CMSIS 库	从队列取一个固定大小副本；返回 osOK 才能使用 block 的新内容。
osKernelGetTickCount / CMSIS 库	读取 RTOS 当前 tick；所有控制超时统一使用同一种计时源。
Command_Validate / 自建	按步骤 05 的规则检查业务范围和两个动作 ID；无效时返回 false。

变量说明：名称含义与保存内容

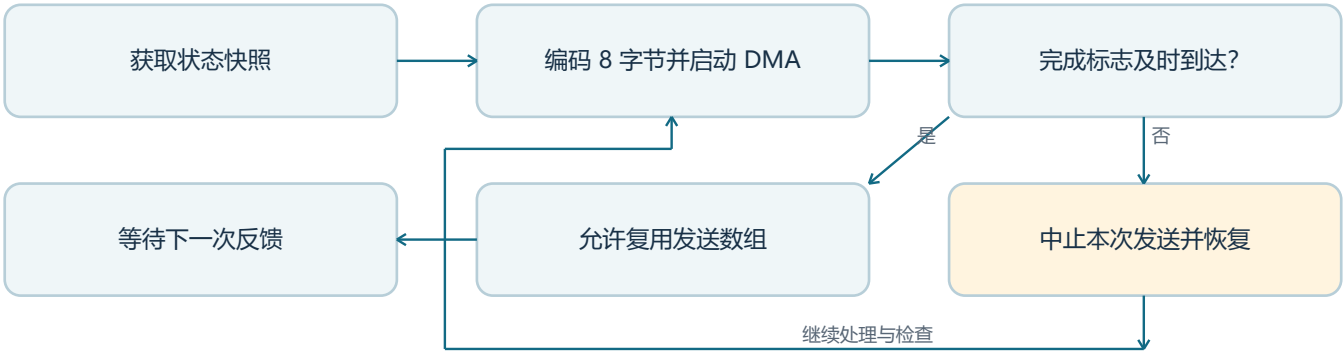
名称与类型	含义、用途与所属模块
parser / Parser	RxTask 私有解析上下文，不能每次循环重新清零，否则无法跨回调拼帧。
block / RxBlock	任务收到的字节块副本；tick 在 ISR 记录，避免排队过久的数据被当成新到。
cmd.tick / uint32_t	完成合法帧所在块的接收 tick；只有帧通过校验后才用于刷新在线状态。
used / uint8_t	窗口中已收字节数量，0～8。

本步验收：通过后再继续

一帧按 3+5 字节分开发送、两帧连续发送、前面加随机噪声、第一帧损坏后跟正确帧：都只把完整有效命令交给控制层。非法数据持续输入时，LED 不应永久保持连接。

08 DMA 发送二进制反馈

本步目的：建立唯一的发送者，保证 DMA 发送期间缓冲区内容不变，避免上一作业那种时好时坏的乱码。



先写打包，再写发送

放 protocol.c，头文件声明 Protocol_Encode。由 TxTask 调用；不在 CAN 中断里直接打包发送。

```
static void WriteI16LE(uint8_t *dst, int16_t value)
{
    uint16_t u = (uint16_t)value;
    dst[0] = (uint8_t)u;
    dst[1] = (uint8_t)(u >> 8);
}

void Protocol_Encode(uint8_t out[8], uint8_t state,
                     int16_t speed, int16_t position)
{
    out[0] = 0xBBU;
    out[1] = 0xAAU;
    out[2] = state;
    WriteI16LE(&out[3], speed);
    WriteI16LE(&out[5], position);
    out[7] = Protocol_Crc8(out, 7U);
}
```

反馈速度和位置字段只有 int16，发送前先检查 isfinite，再四舍五入并限制在 -32768～32767，最后转型；超范围必须按事先约定饱和，不能直接溢出回绕。内部多圈位置继续保持宽范围精度，只有上传时受协议限制。

发送任务一次只允许一个 DMA 事务

回调统一放 uart_tx.c；同名 HAL 回调在整个工程只能有一个强定义。

```
#define TX_DONE (1U << 1)
#define TX_ERROR (1U << 2)
static uint8_t tx_frame[8];

void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1) {
        osThreadFlagsSet(txTaskHandle, TX_DONE);
    }
}
```

TxTask：等待 telemetryQueue → 取出当次状态（队列有积压则取最新快照）→ 编码 tx_frame → osThreadFlagsClear(TX_DONE|TX_ERROR) 清旧通知 → HAL_UART_Transmit_DMA → 等待 TX_DONE 或 TX_ERROR。只有成功完成或可靠中止后，才允许改写 tx_frame。等待上限可先设 20 tick；115200 下 8 字节约 0.694 ms，20 ms 已有很大余量。

TxTask 内的发送片段。标志返回值先判断 osFlagsError，高位错误码不能当作成功置位。

```
osThreadFlagsClear(TX_DONE | TX_ERROR);
if (HAL_UART_Transmit_DMA(&huart1, tx_frame, 8U) == HAL_OK) {
    uint32_t flags = osThreadFlagsWait(
        TX_DONE | TX_ERROR, osFlagsWaitAny, 20U);
    if ((flags & osFlagsError) || (flags & TX_ERROR)) {
        HAL_UART_AbortTransmit(&huart1);
        /* Report the fault; buffer reuse is allowed only after abort. */
    }
} else {
    /* HAL_BUSY/HAL_ERROR: recover and retry on a later cycle. */
}
```

在全工程唯一的 HAL_UART_ErrorCallback 中同时通知接收恢复和发送错误。接收重启只操作 RX；发送中止只操作 TX。正常阶段始终 DMA，故障恢复不通过切回轮询模式来假装成功。不要再让 ControlTask、RxTask 或 printf 直接使用 USART1 发送。

为什么本次不能在同一串口发送中文状态

考核上位机按固定二进制协议解析 USART1。若插入“连接成功”等中文、换行或调试 printf，就破坏了这个数据流。需要文字日志时，使用独立调试串口或调试器变量。普通串口助手必须切到 HEX 显示；二进制字节显示成奇怪字符并不等于串口损坏，也不是改 UTF-8 就能解决。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
Protocol_Encode / 自建	把状态和两个 int16 编码成 8 个协议字节。
HAL_UART_Transmit_DMA / HAL 库	启动非阻塞发送；HAL_OK 只表示启动成功，不代表已经发完。
HAL_UART_TxCpltCallback / 用户实现的 HAL 回调	完成后通知唯一发送任务，不能在这里开始复杂格式化。
osThreadFlagsWait / CMSIS 库	等待匹配位；默认清除已等待的位。最高位为错误时应按错误处理。
HAL_UART_AbortTransmit / HAL 库	任务中用于超时后的发送中止，确认结束后再复用数组。

变量说明：名称含义与保存内容

名称与类型	含义、用途与所属模块
tx_frame / static uint8_t[8]	实际被 DMA 读取的数组，使用静态存储保证持续有效。
TX_DONE / TX_ERROR	第 1/2 位通知；与第 0 位 APP_READY 分开，不用额外创建线程标志对象。
Telemetry	控制任务提交的同一时刻快照；避免发送任务读取一半旧速度、一半新位置。

本步验收：通过后再继续

HEX 看到持续 8 字节 BB AA 帧；CRC 正确，发送节奏约 50 Hz；连续运行时不出现 HAL_BUSY 越积越多。切断串口接收不能把发送 DMA 数组提前覆盖。

09 加入连接判断、LED 与断线急停

本步目的：让通信故障能够直接阻止电机输出，并明确自动重连与恢复运动的区别。



连接状态由什么决定

每次合法、业务有效且未过期的命令到达 ControlTask，才更新 last_cmd_tick，置 seen_cmd=true。主机在线 = 曾收到有效命令，且 now-last_cmd_tick≤HOST_TIMEOUT_TICKS。用 uint32_t 做差，可以正确处理计时器正常回绕；不要比较 now>last+timeout。

ControlTask 中；本教程 tick=1 ms。300 ms 先观察上位机发包周期后调整，通常至少容纳几次丢包。

```
bool host_online = seen_cmd &&
    ((uint32_t)(now - last_cmd_tick) <= 300U);

HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin,
                  host_online ? GPIO_PIN_SET : GPIO_PIN_RESET);
```

LED0 表示主机通信在线，不等于电机已使能。CAN 断线但主机仍持续发包时，LED 可以仍然亮，反馈状态应为 FF。主机只有噪声或 CRC 错包时，不能刷新在线时间。

建议的三个控制状态

状态	允许做什么	转移条件
SAFE / 上电或断线	重复请求零电流；清积分、预设动作和旧命令；LED 按通信有效性显示。	通信恢复且 CAN 稳定，仍不立刻恢复旧目标。
WAIT_ARM / 待确认	继续零电流；只允许用户用速度模式 B=0 确认。	收到新鲜零速命令后进入 READY，当前周期仍输出 0。
READY / 可运动	应用后续速度、位置、动作命令。	任一故障立即回 SAFE。

断线锁存发生后，按命令时间戳丢弃锁存之前的队列数据，防止排队旧命令重新启动电机。队列满、RX 丢字节和 CAN 接收溢出也要进入故障通道，不是只增加一个计数器。可用单独线程标志 COMM_FAULT / CAN_FAULT 通知控制任务；这些标志与 APP_READY 使用不同位。

上位机如果持续重复非零目标，恢复连接后 LED 会亮，但保持零电流。需要在上位机先发零速，再发新目标。这是本教程选择的重启保护，演示时明确说明；题目允许自行设计急停策略。若队里要求自动恢复运动，应单独评估和修改，不要默认沿用旧非零目标。

ControlTask 中故障分支的执行顺序

① 判断故障。② 锁存 SAFE 并取消动作。③ PID_Reset。④ 将本轮 current_cmd 直接置 0。⑤ 向 CAN 发送零电流。⑥ 继续状态检测和反馈。把“目标速度设 0”并不一定等于零电流，因为 PID 可能仍有积分；急停必须旁路控制器。

CAN 发不出去时软件无法保证电调立即收到零命令。还应及时中止旧非零待发邮箱、检查 bus-off，并验证所用电调自身的失联停机行为。上位机断线与 MCU 调试暂停不是同一事件：断点可能让全部任务和超时检查都停止，所以电机带电调试不要把软件急停当成唯一保障。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
Comm_RequestFault / 自建	给 ControlTask 设置通信故障位；可封装 osThreadFlagsSet。控制任务接到后执行完整停机流程。
Control_EnterSafe / 自建	统一清理 PID、动作与使能状态，置电流为 0；不阻塞，不在内部 HAL_Delay。
PID_Reset / 自建，步骤 13	清空积分和微分历史，不改变 PID 参数。
HAL_GPIO_WritePin / HAL 库	根据 host_online 控制 LED0；不是根据“收到任何字节”点亮。

变量说明：名称含义与保存内容

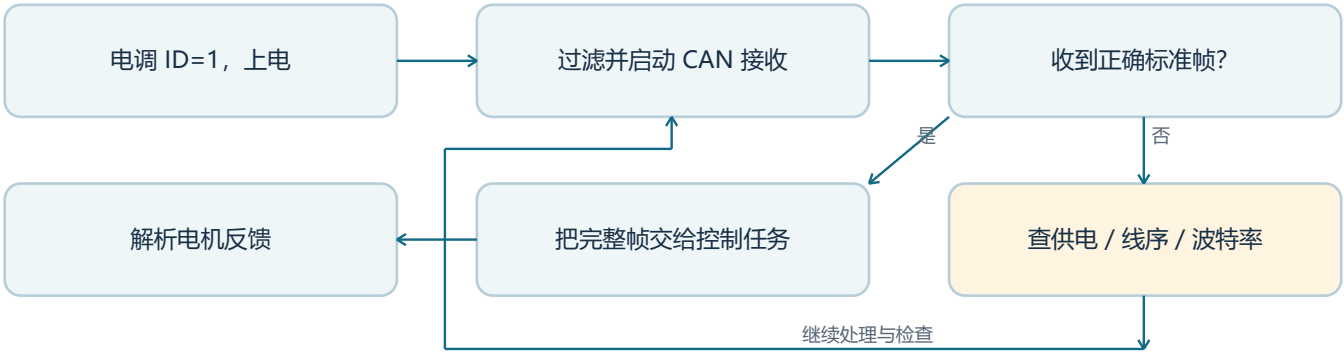
名称与类型	含义、用途与所属模块
seen_cmd / bool	上电初始 false，防止 tick 很小时误判已连接。
last_cmd_tick / uint32_t	最近一次有效命令的时间。
safety_state / 枚举	SAFE、WAIT_ARM、READY 的显式状态，不能删除。
current_cmd / int16_t	本轮将发送的电流原始指令；故障分支强制为 0。

本步验收：通过后再继续

没有上位机时 LED 灭且电流 0；持续正确帧 LED 亮；拔 MX 后在超时内 LED 灭、电流 0；重连 LED 自动亮，但先零速确认才允许运动。持续坏帧不能维持在线。

10 接通 CAN，只接收反馈

本步目的：先证明主控能连续收到正确电调反馈，再考虑发电流。



CAN 协议与串口协议分开处理

项目	手册要求
总线	1 Mbps，标准 11 位 ID，数据帧，DLC=8。
反馈 ID	0x200+电调 ID。教程 ID=1，则反馈为 0x201。
反馈 data[0..1]	转子机械角，0～8191，高字节在前。
反馈 data[2..3]	有符号转子转速 rpm，高字节在前。
反馈 data[4..5]	电流/力矩反馈，含义与缩放按对应电调手册；初版闭环不必依赖它。
反馈 data[6]	C620 为温度；C610 不要按相同温度字段擅自解释。

motor_can.c：配置过滤器、启动与回调

包含 can.h、app_types.h、rtos_handles.h。MotorCan_RequestFault 为自建：给 ControlTask 发故障位；同时在 CAN 错误回调对 FIFO overrun/bus-off 置故障。

```
HAL_StatusTypeDef MotorCan_Start(void)
{
    CAN_FilterTypeDef f = {0};
    f.FilterBank = 0U;
    f.FilterMode = CAN_FILTERMODE_IDMASK;
    f.FilterScale = CAN_FILTERSCALE_32BIT;
    f.FilterIdHigh = 0x201U << 5;
    f.FilterIdLow = 0U;
    f.FilterMaskIdHigh = 0xFFE0U;
    f.FilterMaskIdLow = 0x0006U;
    f.FilterFIFOAssignment = CAN_FILTER_FIFO0;
    f.FilterActivation = ENABLE;
    f.SlaveStartFilterBank = 14U;
    if (HAL_CAN_ConfigFilter(&hcan1, &f) != HAL_OK)
        return HAL_ERROR;
    if (HAL_CAN_Start(&hcan1) != HAL_OK) return HAL_ERROR;
    return HAL_CAN_ActivateNotification(&hcan1,
        CAN_IT_RX_FIFO0_MSG_PENDING | CAN_IT_RX_FIFO0_OVERRUN);
}

void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan)
{
```

```
    if (hcan->Instance != CAN1) return;
    while (HAL_CAN_GetRxFifoFillLevel(hcan, CAN_RX_FIFO0)) {
        CAN_RxHeaderTypeDef header;
        CanFrame frame;
        if (HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0,
                                 &header, frame.data) != HAL_OK)
            break;
        if (header.IDE != CAN_ID_STD || header.RTR != CAN_RTR_DATA
            || header.DLC != 8U || header.StdId != 0x201U)
            continue;
        frame.id = header.StdId;
        frame.tick = osKernelGetTickCount();
        if (osMessageQueuePut(canRxQueueHandle,
                              &frame, 0U, 0U) != osOK) {
            MotorCan_RequestFault();
        }
    }
}
```

在步骤 06 的 InitTask 中增加 MotorCan_Start 成功检查，再发送 APP_READY。过滤器只接受 0x201 标准数据帧。若电调 ID 不是 1，需要同步修改过滤器、回调检查和发送槽位，不能只改一个地方。

不要让 CAN ISR 直接执行 PID，也不要 printf。这里队列保留逐帧反馈，是因为多圈位置展开需要连续角度；只保留最新一帧而丢掉中间许多圈，就无法推回真实转过的圈数。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
HAL_CAN_ConfigFilter / HAL 库	设置硬件过滤器，减少无关帧干扰。
HAL_CAN_Start / HAL 库	把 CAN 控制器从初始化态切到工作态。
HAL_CAN_ActivateNotification / HAL 库	允许指定 CAN 事件产生回调；CubeMX 仅打开 NVIC 还不够。
HAL_CAN_RxFifo0MsgPendingCallback / 用户实现回调	硬件 FIFO 有消息时由 HAL 调用，取帧并复制入队。
HAL_CAN_GetRxMessage / HAL 库	从硬件 FIFO 取出帧头与 8 字节数据，返回 HAL 状态。
MotorCan_RequestFault / 自建	队列溢出、硬件 FIFO 溢出或总线错误时通知控制任务。

变量说明：名称含义与保存内容

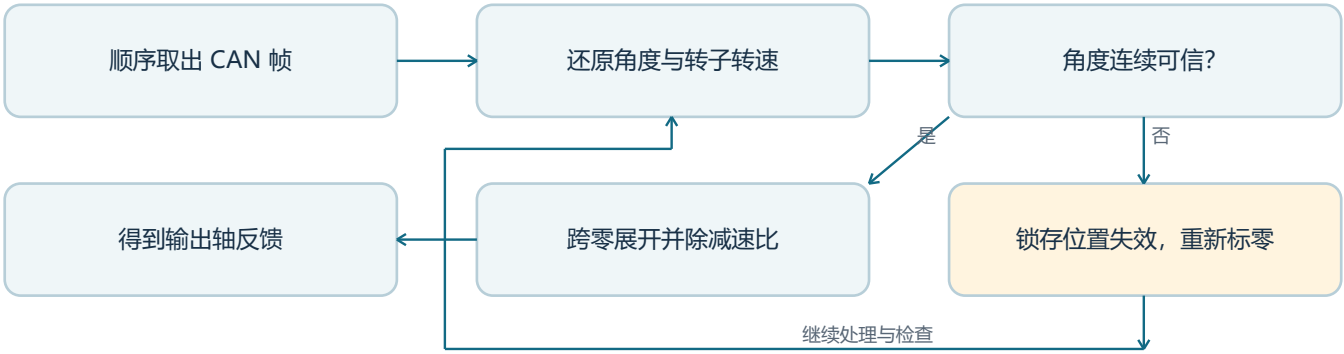
名称与类型	含义、用途与所属模块
f / CAN_FilterTypeDef	临时过滤器配置，库复制设置到硬件寄存器。
header / CAN_RxHeaderTypeDef	保存 ID、帧类型和长度。
frame.tick	接收时间，供反馈新鲜度和丢帧间隔检查。

本步验收：通过后再继续

观察 0x201 反馈连续到达，转动输出轴时 data[0..3] 改变；没有任何非零 CAN 电流命令。收不到时按“电源→ID→CAN 线→终端→1 Mbps→过滤器→中断”顺序检查。

11 计算输出轴速度与多圈位置

本步目的：将电调的转子数据换算成作业要求的减速后 rpm 和 deg。



公式与单位

输出轴速度 = MOTOR_DIR × 转子速度 ÷ gear_ratio。输出轴角度 = MOTOR_DIR × 累计转子编码计数 × 360 ÷ (8192 × gear_ratio)。M2006 的减速比是 36；M3508 的精确减速比是 3591/187。不要把 M3508 简写成 19 后用于要求准确多圈角度的计算。

单帧角度只知道转子当前一圈中的位置。上一帧 8190，这一帧 2，直接相减得到 -8188；实际上是正转了 4 个计数，所以小于 -4096 时加 8192。反向跨零时，大于 4096 就减 8192。

motor_feedback.c/.h；包含 stdint.h、stdbool.h、app_types.h。调用前必须按下文检查反馈连续性；此函数不会凭空恢复丢失的整圈。

```
typedef struct {
    bool initialized;
    uint16_t last_raw;
    int64_t total_count;
    float speed_rpm;
    float position_deg;
    uint32_t last_tick;
} MotorFeedback;

static int16_t ReadI16BE(const uint8_t *p)
{
    uint16_t u = ((uint16_t)p[0] << 8) | p[1];
    return (int16_t)((u < 0x8000U) ? (int32_t)u
                                   : (int32_t)u - 65536);
}

bool MotorFeedback_Update(MotorFeedback *m, const CanFrame *f,
                          float ratio, float direction)
{
    uint16_t raw = ((uint16_t)f->data[0] << 8) | f->data[1];
    if (raw >= 8192U || ratio <= 0.0f) return false;
    if (!m->initialized) {
        m->initialized = true;
        m->last_raw = raw;
        m->total_count = 0;
    } else {
        int32_t delta = (int32_t)raw - m->last_raw;
        if (delta == 4096 || delta == -4096) return false;
        if (delta > 4096) delta -= 8192;
        else if (delta < -4096) delta += 8192;
        m->total_count += delta;
        m->last_raw = raw;
    }
    m->speed_rpm = direction * ReadI16BE(&f->data[2]) / ratio;
    m->position_deg = direction * (float)m->total_count
```

```
                                * 360.0f / (8192.0f * ratio);
    m->last_tick = f->tick;
    return true;
}
```

必须理解的两个边界

第一，首次有效反馈作为 0 度只在“电机上电到收到首帧期间保持静止”时等价于题目的上电零点。若电机先转过再启动主控，软件不知道这段角度。演示时统一上电、等待初始化后再运动。CAN 丢失后轴可能转过多圈，重连不能自动恢复之前的绝对位置；应锁存位置无效，停止动作，按约定回零或重新上电标定。

第二，跨零展开假设两次有效采样间转子转动不足半圈。若最大转子速度为 n rpm，间隔必须小于 30/n 秒。例如 n=6000 时小于 5 ms。CAN“50 ms 才判离线”不能作为角度可靠标准；应另外根据验证过的最大转速设更严格的采样间隔限制。队列溢出、间隔过大或出现无法判定的跳变，就禁止继续位置闭环。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
MotorFeedback_Update / 自建	逐帧更新累计计数和输出轴反馈；输入减速比、方向。
ReadI16BE / 自建 static	把电调大端两个字节转回有符号转速，与串口小端函数分开。

变量说明：名称含义与保存内容

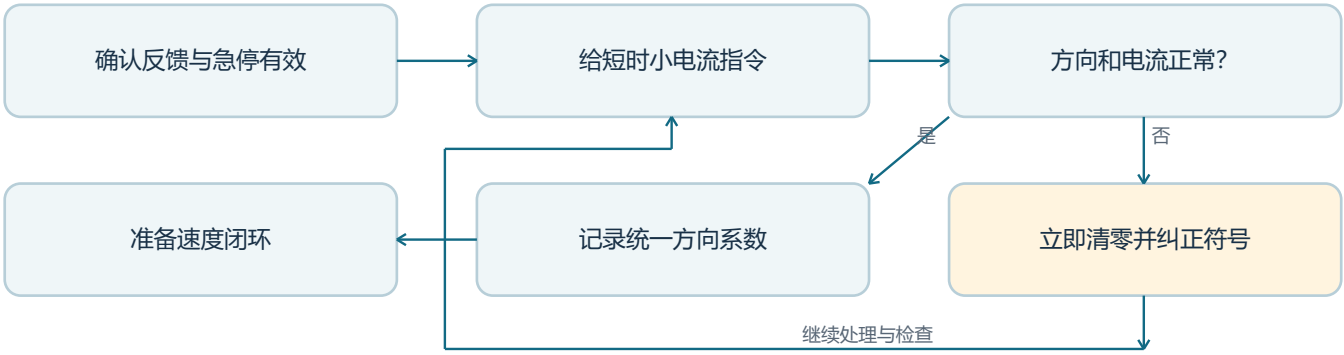
名称与类型	含义、用途与所属模块
last_raw / uint16_t	上一帧的单圈转子角度计数。
delta / int32_t	校正过跨零的相邻计数增量。
total_count / int64_t	从首次有效帧累计的转子计数，用宽整数避免长时间运行溢出。
position_valid / bool，另设	表示多圈展开是否仍可信；断帧过久应锁存 false，不靠新来一帧自动恢复。

本步验收：通过后再继续

断电或保持电流 0 时手动缓慢转输出轴一整圈，位置约变化 360°，不是 360°/减速比；顺时针速度和位置都为正；跨过转子编码器零点时位置连续。

12 用小电流验证方向和限幅

本步目的：把安全限流放到最终 CAN 出口，先确认负反馈方向，再开启 PID。



先区分“电流原始计数”和“安培”

电调	手册缩放	本题硬上限	对应约值
C620	16384 对应 20 A	abs(raw) ≤ 8000	约 9.77 A
C610	10000 对应 10 A	abs(raw) ≤ 4000	4 A

考核文字把 C610 的 4000 近似写成 5 A，但随附手册的缩放是 4000→4 A。本教程遵守明确的原始指令上限 4000，不据“5 A 左右”反推为 5000。上位机 A 字段若是 mA，C620 要按 16384/20000 换算成计数；C610 计数与 mA 数值对应。字段单位必须通过上位机实测确认。

CAN 控制帧

0x200 控制 ID 1～4，0x1FF 控制 ID 5～8。每个电机占两个字节，高字节先发。教程只接 ID=1，因此 0x200 的 data[0..1] 是电机指令，其余 6 字节清零。若同一组有多电机，必须由一个统一发送模块把所有槽位一起打包，不能各任务各发一份并把其他电机清零。

motor_can.c 中的单电机发送函数。要求 hard_limit 来自固定配置，direction 只能 ±1；仅允许控制任务调用。数据会复制进 CAN 邮箱，和 UART DMA 持有数组指针不同。

```
static int16_t LimitCurrent(int32_t raw, int32_t limit)
{
    if (raw > limit) raw = limit;
    if (raw < -limit) raw = -limit;
    return (int16_t)raw;
}

HAL_StatusTypeDef MotorCan_SendCurrent(int16_t requested,
                                       int16_t hard_limit,
                                       int8_t direction)
{
    uint8_t data[8] = {0};
    CAN_TxHeaderTypeDef header = {0};
    uint32_t mailbox;
    int16_t raw = LimitCurrent((int32_t)requested * direction,
                               hard_limit);
    uint16_t u = (uint16_t)raw;
    data[0] = (uint8_t)(u >> 8);
    data[1] = (uint8_t)u;
    header.StdId = 0x200U;
    header.IDE = CAN_ID_STD;
    header.RTR = CAN_RTR_DATA;
```

```
    header.DLC = 8U;
    if (HAL_CAN_GetTxMailboxesFreeLevel(&hcan1) != 3U)
        return HAL_BUSY;
    return HAL_CAN_AddTxMessage(&hcan1, &header, data, &mailbox);
}
```

这里要求三个邮箱全空才装新指令，是为了不让多条旧电流排队。HAL_OK 仅表示已装邮箱，并不保证总线已成功发出。ControlTask 需要计时监控邮箱持续忙的情况；故障时 HAL_CAN_AbortTxRequest 中止旧待发请求，再在中止完成后装入零电流。bus-off 时保持软件安全态，等待总线恢复后仍先发零，不能重发旧目标。

测试时怎么给值

先用数百计数以内的短脉冲作为调试起点，依据实物负载逐渐调整；不是一上来给 8000。每个测试脉冲到时都强制清零。若不转，先排除机械卡阻、电源和 CAN，不要盲目增加电流。记录“正电流→顺时针→正速度”的关系，必要时统一改变 MOTOR_DIR；只反转反馈而不反转电流会把速度环变成正反馈。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
MotorCan_SendCurrent / 自建	执行最终硬限幅和方向变换，打包单电机 CAN 命令。
HAL_CAN_AddTxMessage / HAL 库	把帧写入可用发送邮箱；返回启动结果。
HAL_CAN_GetTxMailboxesFreeLevel / HAL 库	读取空闲邮箱数，防止陈旧指令排队。
HAL_CAN_AbortTxRequest / HAL 库	取消指定邮箱待发请求；安全逻辑还要确认取消完成并处理总线故障。

变量说明：名称含义与保存内容

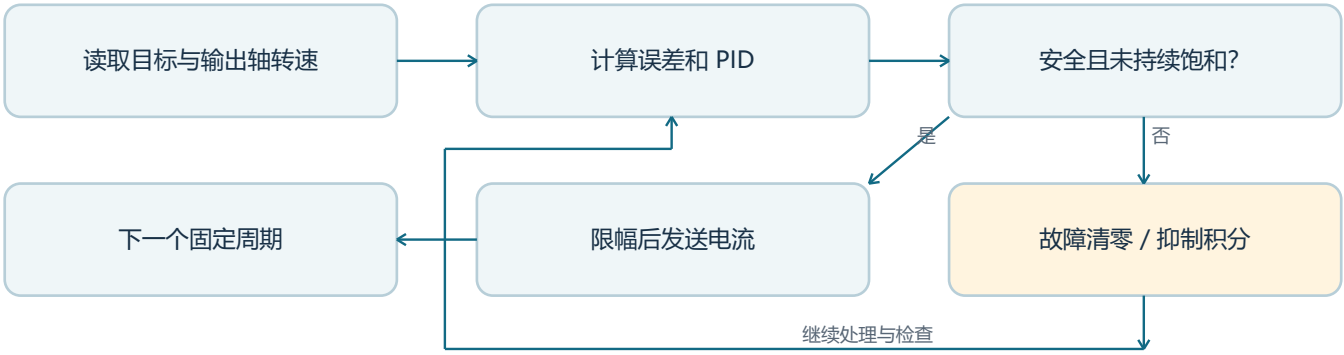
名称与类型	含义、用途与所属模块
hard_limit / int16_t	电调硬上限，仅来自编译配置，不允许上位机提高它。
effective_limit	min(硬上限，当前模式限流，上位机允许值，调试限流)。
mailbox / uint32_t	HAL 返回的邮箱位标识；可用于状态跟踪/中止，不是 CAN ID。

本步验收：通过后再继续

小电流短脉冲方向与测量方向一致；到时恢复零；任何路径下 abs(raw) 都不能越过 8000/4000。拔掉上位机和断开 CAN 时不会继续排队旧非零命令。

13 实现位置式 PID 速度环

本步目的：让目标 rpm 和实际 rpm 的误差通过控制器变成电流，并先完成 200/0 rpm 方波考核。



“位置式 PID”不是“位置控制”

位置式 PID 指本轮直接计算完整输出 u，而不是计算输出增量。这里输入是速度误差，输出仍然是电流。离散形式：error=target-measured；P=kp×error；I 累加 ki×error×dt；D 可用 -kd×(measured-last_measurement)/dt，减小目标阶跃造成的微分突变。输出=P+I+D。

新建 pid.h / pid.c

包含 stdbool.h、math.h；kp/ki/kd 在初始化时须保证有限且非负。这是位置式 PID，采用条件积分抗饱和。

```
typedef struct {
    float kp, ki, kd;
    float integral;
    float last_measurement;
    bool initialized;
} PID;

static float Clamp(float x, float lo, float hi)
{
    return x < lo ? lo : (x > hi ? hi : x);
}

void PID_Reset(PID *p)
{
    p->integral = 0.0f;
    p->last_measurement = 0.0f;
    p->initialized = false;
}

float PID_Update(PID *p, float target, float measured,
                 float dt, float limit)
{
    if (!isfinite(target) || !isfinite(measured) ||
        !isfinite(dt) || !isfinite(limit) ||
        dt <= 0.0f || limit <= 0.0f) {
        PID_Reset(p);
        return 0.0f;
    }
    float error = target - measured;
    float d = p->initialized
        ? -p->kd * (measured - p->last_measurement) / dt : 0.0f;
    float candidate = Clamp(p->integral + p->ki * error * dt,
                            -limit, limit);
    float trial = p->kp * error + candidate + d;
    bool pushes_high = trial > limit && error > 0.0f;
    bool pushes_low = trial < -limit && error < 0.0f;
```

```
    if (!pushes_high && !pushes_low) p->integral = candidate;
    p->integral = Clamp(p->integral, -limit, limit);
    p->last_measurement = measured;
    p->initialized = true;
    return Clamp(p->kp * error + p->integral + d,
                 -limit, limit);
}
```

控制周期和测试信号

ControlTask 每 1 tick 执行一次。先记录 next=osKernelGetTickCount()，每轮 next+=1，再 osDelayUntil(next)。用实际 elapsed ticks ×0.001 得到 dt；若超时过多（例如多毫秒），记录超期并进入安全处理，而不是连续快速追赶并假装每次都是 1 ms。不要在控制任务中等待串口发完、HAL_Delay 或等待很长的队列超时。

方波信号仅在明确的速度测试模式启用；HOST/CAN 故障和限流仍优先，不能为测试绕过急停。

```
uint32_t elapsed = (uint32_t)(now - test_start_tick);
float target_rpm = ((elapsed / 1000U) % 2U == 0U)
                 ? 200.0f : 0.0f;
float current = PID_Update(&speed_pid, target_rpm,
                           motor.speed_rpm, dt, effective_limit);
```

先 ki=0、kd=0，从较小 kp 和低电流限幅开始，观察响应方向和振荡；再逐渐加入 ki 消除稳定误差；只有确有需要时再加 kd，并考虑测量噪声滤波。不能从网上抄一组固定 PID 参数就声称适合两种不同电机和负载。用相同采样周期记录目标/实际速度和电流，依据波形调整。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
PID_Update / 自建	输入目标、反馈、dt 秒和输出上限，返回本轮完整控制量；速度环返回电流计数。
PID_Reset / 自建	急停、模式切换、反馈失效时清动态状态；普通相同心跳帧不能每次清零。
Clamp / 自建 static	把数值限制到区间，供积分和最终输出共同使用。
osDelayUntil / CMSIS 库	按绝对 tick 等到下个周期；不是“延迟这么多 tick”的 osDelay。
isfinite / C 数学库宏	排除 NaN 和无穷大，避免把异常浮点结果转成电流整数。

变量说明：名称含义与保存内容

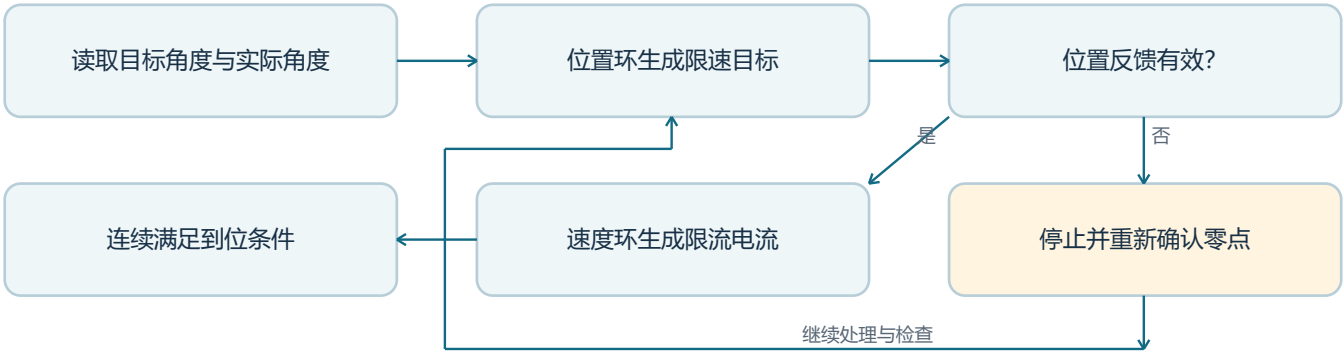
名称与类型	含义、用途与所属模块
kp / ki / kd	比例、积分、微分系数；单位随输入输出决定，改变 dt 写法时不能原样套参数。
integral / float	积分项的累计输出，属于每个 PID 实例。
dt / float	实际控制周期，单位秒；1 ms=0.001f。
candidate / trial	暂定积分和暂定输出；判断是否会继续推动饱和后再决定是否保存。

本步验收：通过后再继续

先低速稳定再做 200/0 rpm 每秒交替；负载变化后能恢复目标；电流始终受限，长期限幅后积分不会越积越大。记录实际波形，不能只凭“电机在转”判定通过。

14 用位置外环和速度内环构成串级

本步目的：位置误差先变为目标 rpm，再由已经验证过的速度环产生电流。



两层闭环各负责什么

位置外环：target_deg - position_deg → target_rpm。速度内环：target_rpm - speed_rpm → current_raw。位置 PID 输出单位是 rpm，因此最大输出速度 A 正好作为外环输出限幅；内环输出单位是电流计数，仍有独立的硬限流和调试限流。

ControlTask 内位置模式分支；同一个 speed_pid 用于速度内环，position_pid 是独立实例。

```
float speed_target = PID_Update(&position_pid,
    target_deg, motor.position_deg, dt, max_speed_rpm);

float current = PID_Update(&speed_pid,
    speed_target, motor.speed_rpm, dt, effective_current_limit);
```

最容易开始的外环是只开 kp，ki=kd=0。位置环通常比速度内环慢，先以每 5 ms 更新外环、每 1 ms 更新内环为起点；外环 dt 必须是其真实更新间隔，目标转速在两次外环更新之间保持。上方片段如果每 1 ms 都执行两环，也必须使用各自真实 dt；不要写成实际 5 ms 却仍传 0.001。

为什么不能用角度完全相等判断到位

真实反馈存在量化与抖动，浮点数也不应依赖完全相等。起步选择 abs(position_error)<2° 且 abs(speed)<3 rpm，连续保持 100 ms 才记为到位。阈值是设计选择，需要按实物精度调整。进入范围开始记时，离开范围立即清除计时；不能把分散的多个 10 ms 累加成“连续 100 ms”。

到位后仍继续位置闭环，才能在外力扰动时维持位置。位置到位不是立刻结束任务或切零电流；只有安全故障分支才直接零电流。速度模式的到达判断类似，可用速度误差小于 5 rpm 连续 100 ms，独立于位置误差。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
PositionControl_Update / 建议自建	按外环节拍更新速度目标，再调用速度 PID；也可先直接写在控制任务的清晰分支里。
Arrival_Update / 建议自建	输入误差与速度，更新连续到位计时；返回是否稳定到位。
fabsf / C 数学库	求 float 绝对值；不要用整数 abs 处理角度小数。

变量说明：名称含义与保存内容

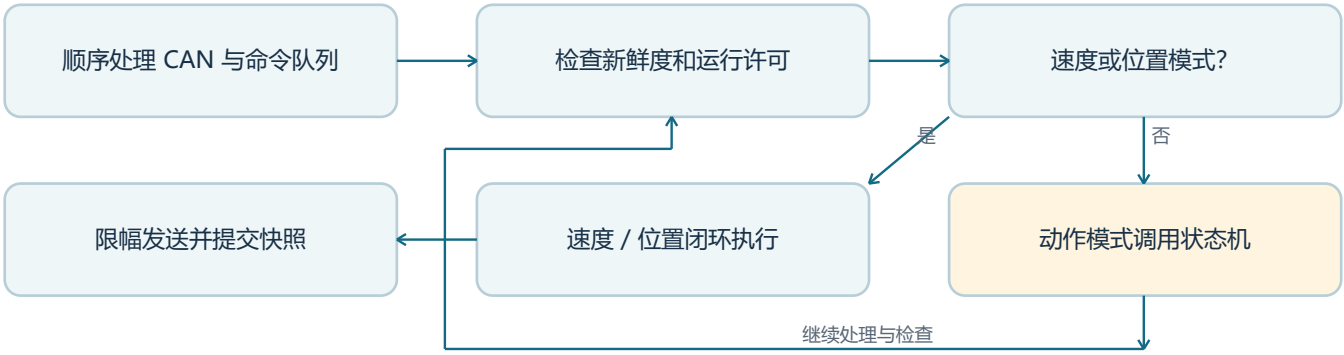
名称与类型	含义、用途与所属模块
position_pid / PID	位置外环私有积分/历史；不可与速度环共用同一个 PID 结构体。
max_speed_rpm / float	位置模式 A 提供的输出轴速度限幅；负值拒绝、零值禁止运动。
in_band_since / uint32_t	首次连续进入容差区的 tick，配合独立 bool 记录是否正在计时。
arrived / bool	连续条件已满足，用于状态上传和动作切换。

本步验收：通过后再继续

按 0→90→0→720°逐级测试；多圈目标不被取模成一圈。速度不会超过设定上限，未到位不提前报告完成；扰动后仍能恢复目标。

15 把串口命令接入唯一的控制任务

本步目的：把已单独验证的通信和控制连接起来，避免多个任务同时修改目标、PID 或电流。



数据由谁拥有

数据	唯一写入者	其他模块怎么使用
DMA 原始缓冲区	硬件 DMA	ISR 复制到 RxBlock。
解析器状态	RxTask	只通过 Command 队列交付命令。
motor、mode、target、PID、preset	ControlTask	TxTask 只读取 Telemetry 副本，不直接读这些全局结构。
tx_frame 与发送事务	TxTask	完成 ISR 只发送通知。

先写控制循环的骨架，再把已验证函数放进去

这是顺序伪代码，用于组织 control_task.c；这些英文动作名不是现成库函数，不能直接复制当成完整实现。

```
/* CONTROL FLOW PSEUDOCODE - implement using steps 09 to 16. */
wait_for_APP_READY();
initialize_state_and_PID();
for (;;) {
    now = osKernelGetTickCount();
    drain_CAN_queue_in_order();
    drain_fresh_commands_in_order();
    check_host_CAN_and_timing_faults();
    update_connection_LED();

    if (!motion_allowed) {
        reset_controllers_and_cancel_action();
        current_cmd = 0;
    } else {
        update_mode_or_preset_targets();
        current_cmd = run_selected_control_loop();
    }
    send_limited_current_with_mailbox_monitor();
    publish_telemetry_every_20_ms();
    sleep_until_next_control_period();
}
```

控制任务先更新实际反馈，再更新目标，然后计算输出。命令 A/B 内容相同、仅时间戳变化，属于心跳，不能每次 PID_Reset，否则积分永远建立不起来。模式真正切换时才重置控制器、到位计时和旧动作；目标变化时更新目标并重置到位计时，积分是否清零按控制设计处理。

模式 00：A→电流上限，B→目标 rpm。模式 11：A→速度上限，B→目标 deg，内环电流上限由本地配置提供。模式 22：四字节 action_id→动作选择，不能继续把 A 解释成电流上限。位置模式 A=0 在本教程按禁止输出处理，进入零电流分支；不要让外环目标速度为 0 后仍因内环积分输出电流。

状态字节的选择

条件	S	解释
CAN 从未收到 / 反馈超时	FF	考核定义的电机异常。
速度模式，未稳定到速	AA	A0 | 0A。
速度模式，稳定到速	AF	A0 | 0F。
位置模式，尚未稳定到位	BA	B0 | 0A。
位置模式，已稳定到位	BF	B0 | 0F。

“AA”作为状态值与帧头中的 AA 恰好相同，这是允许的；解析必须同时检查帧位置和 CRC，不能只搜索任意 AA。预设速度动作的高半字节使用 A0，预设位置动作使用 B0，是本教程对题目未单列预设高半字节的解释，需要与上位机显示验证。动作没全部完成时低半字节一直为 0A。

题目没有给主机离线、待使能和“角度已失效但 CAN 又有帧”专门状态码，不要擅自造一个并称为标准协议。保留本地故障变量；协议反馈按既定状态语义与上位机商定扩展。CAN 不在线时优先发 FF。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
ControlTask / 自建任务入口	持有全部运动状态，执行周期循环；由 RTOS 调度，不由 main 反复调用。
Control_ApplyCommand / 建议自建	比较模式与目标是否真正改变，更新限幅/目标，处理零速使能与动作启动。
Control_BuildTelemetry / 建议自建	从当前同一份状态创建 Telemetry，决定状态字节并入队。
osMessageQueuePut / CMSIS 库	把快照复制到 telemetryQueue，控制任务使用 timeout=0；满时丢弃本次快照或记录，不允许阻塞电机环。

变量说明：名称含义与保存内容

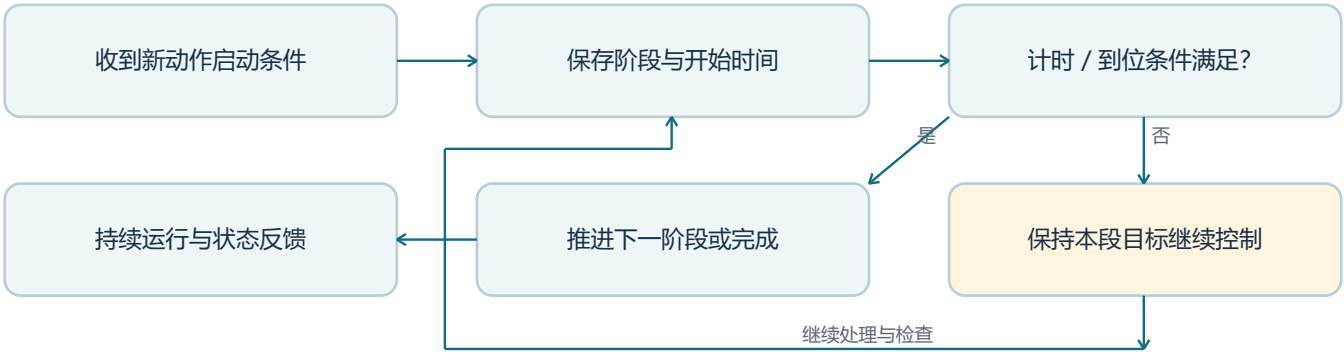
名称与类型	含义、用途与所属模块
mode / uint8_t	当前 00/11/22 模式。
target_rpm / target_deg	分别是期望输出轴速度和位置。
last_command	判断重复心跳；比较有效载荷，不比较每次都会变的 tick。
next_report_tick	下一次 50 Hz 状态快照时间，与 1 kHz 控制节拍独立。

本步验收：通过后再继续

速度模式能正反转，位置模式能到指定多圈位置；改变 A 的限制有实际效果；重复心跳不重置 PID；发送状态与实际到达情况一致。

16 以状态机执行两组预设动作

本步目的：不用几秒钟的阻塞延时，保证每个动作阶段仍然维持控制、反馈和急停。



动作一：0x00000000，按时间切换速度

阶段	目标 rpm	持续时间	相对动作开始区间
0	60	1 s	0 ~ 1 s
1	120	1 s	1 ~ 2 s
2	180	1 s	2 ~ 3 s
3	240	2 s	3 ~ 5 s
4	300	2 s	5 ~ 7 s
5	150	1 s	7 ~ 8 s
6	0	1 s	8 ~ 9 s

这一动作持续时间从各段目标开始施加时计算，不必等达到速度后才开始计时；总时长 9 s。阶段切换只改变速度目标，由同一速度 PID 继续控制。最后保持目标 0，标记动作完成。控制超期导致跨过重要阶段时应按故障处理，而非快速补发一串目标。

preset.c：返回 true 表示整个速度动作结束；elapsed_ms 是从本次动作起点计算的毫秒数。

```
typedef struct {
    float rpm;
    uint32_t duration_ms;
} SpeedSegment;

static const SpeedSegment speed_segments[] = {
    {60.0f, 1000U}, {120.0f, 1000U}, {180.0f, 1000U},
    {240.0f, 2000U}, {300.0f, 2000U}, {150.0f, 1000U},
    {0.0f, 1000U}
};

bool Preset_SpeedAt(uint32_t elapsed_ms, float *target)
{
    for (uint32_t i = 0U; i < 7U; ++i) {
        if (elapsed_ms < speed_segments[i].duration_ms) {
            *target = speed_segments[i].rpm;
            return false;
        }
        elapsed_ms -= speed_segments[i].duration_ms;
```

```
    }
    *target = 0.0f;
    return true;
}
```

动作二：0x11111111，按到位和停留切换位置

阶段	目标	进入下一阶段条件
MOVE_0	0°	连续满足到位条件后去 720°，没有额外初始停留要求。
MOVE_720	720°	稳定到位后开始 3 s 停留。
HOLD_720	保持 720°	连续处于到位范围 3 s 后去 360°；明显离开范围则重新等待到位并计时。
MOVE_360	360°	稳定到位后开始 1 s 停留。
HOLD_360	保持 360°	连续保持 1 s 后去 0°。
MOVE_END_0	0°	稳定到位后动作完成，继续位置保持。

不能在发出 720°目标的那一刻就启动 3 s 停留，否则运动时间会吞掉本该停留的时间。MOVE 和 HOLD 都继续执行位置外环和速度内环。HOLD 仅改变什么时候进入下一个阶段，并不暂停任务。

状态机片段，按上表补齐其他四个阶段；state 等变量保存在 Preset 上下文。Arrival_Reset 自建，用于新目标重新进行到位判断。

```
/* Example transition inside Preset_PositionUpdate(). */
case MOVE_720:
    *target_deg = 720.0f;
    if (arrived) {
        state = HOLD_720;
        hold_start_tick = now;
    }
    break;

case HOLD_720:
    *target_deg = 720.0f;
    if (!arrived) {
        state = MOVE_720;
    } else if ((uint32_t)(now - hold_start_tick) >= 3000U) {
        state = MOVE_360;
        Arrival_Reset();
    }
    break;
```

避免每个心跳都把动作从头启动

上位机定时发送同一个动作 ID，只应更新在线时间，不应反复 Preset_Start。第一次从普通模式进入动作模式、或动作 ID 真正变化，才启动一次；结束后重复相同 ID 也不自动再播。若想再执行同一个动作，先切普通模式（可发零速），再切回动作模式。题目没有序号或独立“启动”位，因此无法区分两个完全相同的帧究竟是重发还是用户再次点击。

任意阶段出现主机断线、CAN 故障或角度失效，立即 Preset_Cancel，进入安全态。重连后不从原阶段继续。测试时尤其检查动作停留中拔线，避免只在阶段切换时才检查故障。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
Preset_Start / 自建	记录 action_id、开始时间，初始化速度/位置动作的阶段。只在新动作事件调用一次。
Preset_SpeedAt / 自建	按经过时间查表得到速度目标；不阻塞，不直接发 CAN。
Preset_PositionUpdate / 自建	输入当前时间、到位状态，更新阶段并输出位置目标。
Preset_Cancel / 自建	清运行状态；实际电流归零由统一安全分支保证。
Arrival_Reset / 自建	清连续到位计时，防止旧目标的 arrived 直接用于新目标。

变量说明：名称含义与保存内容

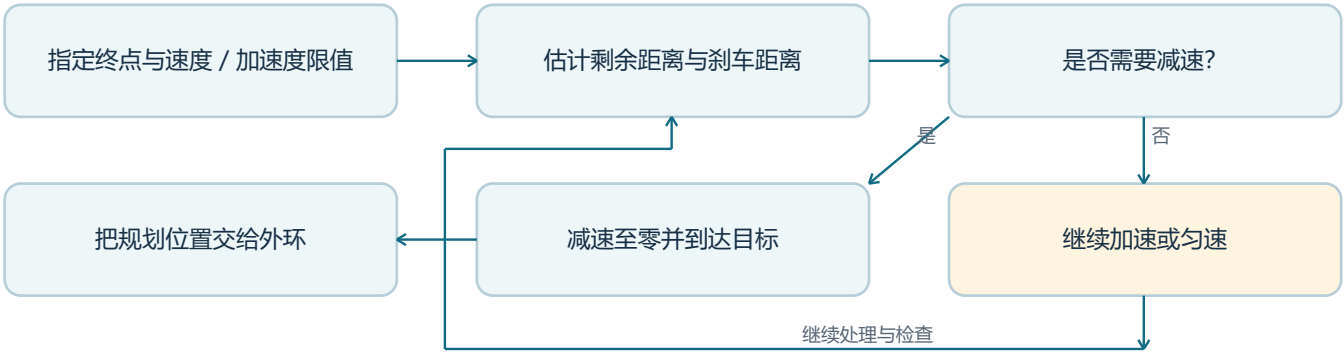
名称与类型	含义、用途与所属模块
action_id / uint32_t	0x00000000 或 0x11111111。
action_start_tick	速度动作的整体开始时间；重复心跳不更新。
hold_start_tick	位置实际到位后开始停留的时间。
running / done	运行与完成标志；动作子阶段到位不等于整个动作完成。

本步验收：通过后再继续

速度动作总计约 9 s；位置动作在 720°真正到位后停 3 s、360°到位后停 1 s；重复动作帧不反复重启；停留中拔线也能急停。

17 选做：给位置目标加入梯形速度规划

本步目的：在基本位置闭环合格后，限制加速度和最大速度，减少目标突变。



这一步解决什么问题

直接把位置从 0 改成 720°，外环可能立即给最大转速。轨迹规划每个周期生成一个逐渐前进的参考位置 ref_pos，并限制参考速度 ref_vel 和加速度 a_max。电机仍靠双环跟踪它；规划器不代替 PID，也不绕过真实反馈。

统一使用 deg、deg/s、deg/s²。rpm 转成 deg/s 要乘 6。估算刹车距离 d_stop=ref_vel²/(2×a_max)。离目标还远时加速，到最大速度匀速，剩余距离接近刹车距离时减速；短距离可能只有三角形速度曲线，没有匀速段。

实际实施顺序

① 先固定单方向的静止到静止移动。② 用 dt 积分 ref_vel 和 ref_pos，限制本周期增量不越过终点。③ 把 ref_pos 送给位置环，而不是直接把 ref_vel 当电流。④ 到终点后 ref_vel=0、ref_pos=target_deg。⑤ 最后再处理运动中换目标、反向、初始速度非零和故障恢复，这些情况需要重新规划。

不要仅凭某个时刻 d_stop 大于距离就做一个没有状态的加减速开关，否则可能在临界处反复抖动。将加速、巡航、减速、完成保存为显式阶段；使用真实 dt，并在急停后清规划状态。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
Trajectory_Start / 自建	从当前测量位置和约定初速度开始规划，保存目标与限制。
Trajectory_Update / 自建	每周期输入 dt，输出 ref_pos/ref_vel；处理加速、减速与终点。
Trajectory_Reset / 自建	故障或重新标零时清轨迹，避免恢复后接着追旧参考。

变量说明：名称含义与保存内容

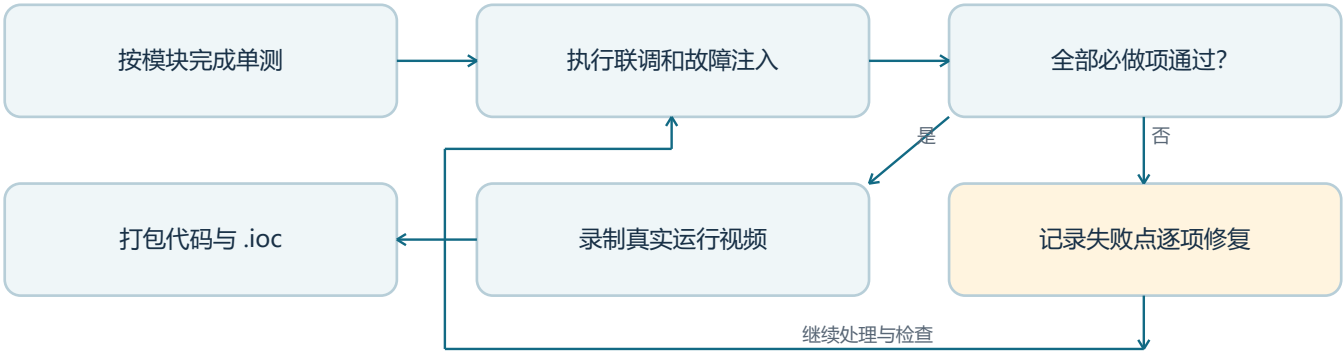
名称与类型	含义、用途与所属模块
ref_pos / float	计划轨迹的位置，deg；不同于真实反馈 position_deg。
ref_vel / float	计划速度，deg/s；输入 PID 前注意单位。
a_max / float	允许加速度，deg/s²，必须大于 0。
d_stop / float	在当前参考速度下的估算停车距离，deg。

本步验收：通过后再继续

不改变最终目标和到位判断；记录参考/实际速度，能看到有限加速度。此项为选做，前面必做功能尚未稳定时不要先调它。

18 联调、排错并准备提交

本步目的：用可重复测试证明满足要求，同时让工程移动后仍能重新构建。



完整验收清单

测试	通过标准	建议记录
串口正确帧 / 拆包 / 粘包	合法命令识别一次；坏帧拒绝；噪声后恢复。	HEX 帧与解析变量。
双向 DMA	稳定接收循环 DMA，发送 DMA；长时间无溢出。	DMA 配置、回调与队列峰值。
拔 MX / 重连	超时内急停，LED 灭；重连 LED 亮，零速确认后允许新运动。	LED 和电机同时入镜。
CAN 断开 / ID 错误	输出安全，反馈 S=FF，不假装达到目标。	HEX 状态和故障变量。
手动转轴	360°输出轴转动对应约 360°；跨零连续。	位置曲线或变量。
速度方波	200/0 rpm 每秒切换，反馈合理跟随，无持续振荡。	目标/实际速度、电流。
位置闭环	正负、多圈位置能稳定到位，速度与电流受限。	目标/实际位置。
两组动作	9 s 速度序列；720°停3 s，360°停1 s，最后回0。	目标、反馈、阶段和时间。
心跳重复与故障	重复动作帧不重启；任何阶段故障都清输出。	动作期间持续帧与拔线测试。

常见错误按现象定位

现象	优先检查
arm-none-eabi-gcc not found	当前 VS Code 进程 PATH 是否包含 Arm 工具链 bin；where.exe arm-none-eabi-gcc。修改用户 PATH 后完整重启 VS Code，或在工具链文件显式配置已安装编译器路径。CMake 本身不等于编译器。
改文件夹后 compile_commands.json 找不到	清理当前工程生成的 build 缓存并重新配置，检查 CMakePresets、VS Code 设置、Ozone ELF 路径是否写死旧目录；不要删除源代码。
undefined reference to 自建函数	实现 .c 是否编译、名称是否一致、是否误写 static、是否混用 C/C++ 但未 extern "C"。
队列创建失败 / 运行到一半挂起	检查句柄、heap 剩余、任务栈水位；RTOS 总堆与单任务栈分别处理。不要只盲目增加所有内存。
DMA 发送完成回调不来	USART 全局中断是否开、DMA IRQ 是否正确链接 HAL、缓冲区是否在 SRAM、是否重复发送导致 BUSY。
串口乱码但 HEX 完整	本次是二进制协议；不要用文本编码判断。若 HEX 丢字节，再查波特率、回调增量、数组寿命和并发发送。
速度或位置差约几十倍	混用了转子/输出轴单位，减速比错误，或反馈端序反了。
一开 PID 就冲转	立即停机；检查反馈符号与命令方向、rpm 单位、dt、限流，不先加大参数。

可移动的 CMake 配置

CMakePresets.json 用 ${sourceDir}/build/${presetName} 作为 binaryDir；工具链文件用 ${sourceDir}/cmake/gcc-arm-none-eabi.cmake。生成的 CMakeCache.txt 和 compile_commands.json 含绝对路径，项目移动后需要在新位置重新配置，它们不属于必须搬运的源文件。工具链的安装路径可固定，工程路径应尽量相对化。

VS Code 在新的工程根目录打开后，先确认没有正在配置的操作，再执行 CMake: Delete Cache and Reconfigure。ConfigureInProgress 表示另一个配置仍在进行，先停止或等待该操作，不要连点。Ozone 选择新工程的 ELF，修改工程里的旧源码映射/路径，然后重载程序。

调试时看哪些变量

在 Ozone 的 Watch/Expressions 中输入变量或表达式，具体窗口名称随版本略有不同。为了容易观察，可暂时把控制状态放在文件作用域的静态结构体，并用 Debug、低优化编译；局部变量在作用域外或优化后可能看不到。不要靠长期断点测试真实 PID 时序：CPU 一停，反馈队列和控制输出行为都变了。

建议观察	用途
host_online、last_cmd_tick、safety_state	区分通信在线与允许运动。
motor.speed_rpm、motor.position_deg、position_valid	验证反馈换算与位置可信度。
target_rpm、target_deg、current_cmd	检查目标到电流的链路。
speed_pid.integral、preset.state	检查积分饱和和动作阶段。
各队列峰值、任务栈剩余、控制超期计数	诊断实时性和内存余量。

uart_send_ok++ 一类纯统计计数在验证后可以删，但 last_cmd_tick、DMA 完成标志、位置有效标志、PID 积分、动作阶段不是“调试变量”，删除会破坏功能。计数器是否删，取决于它有没有被控制逻辑读取。

提交材料

新建提交目录，放完整可重新生成/编译的工程：App、Core、Drivers、Middlewares、.ioc、CMake 配置、工具链文件、链接脚本和必要启动文件。附一份简短 README：板版本、引脚、电调 ID、型号、工具链版本、波特率、协议假设、启停步骤与已验证参数。可附 ELF 便于复现，但不能只交 ELF；build 缓存通常可不交。

录制真实演示视频，依次展示连接与断线、反馈、速度方波、位置控制、两组动作；没有验证的功能不要声称完成。按考核要求把代码、.ioc 和视频提交至指定邮箱 HITwtrobot@163.com。本教程不会代你发送邮件或把教学文档冒充实测报告。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
uxTaskGetStackHighWaterMark / FreeRTOS 原生库	测线程历史最小剩余栈，Cortex-M 常按 StackType_t 项数计，核对当前端口单位；需要相应配置开启。
xPortGetFreeHeapSize / FreeRTOS 堆实现	看 RTOS 堆当前剩余，帮助判断对象创建失败。
本步无新的运动函数	重点是验证和记录，不在提交前随意重构已经稳定的控制链。

变量说明：名称含义与保存内容

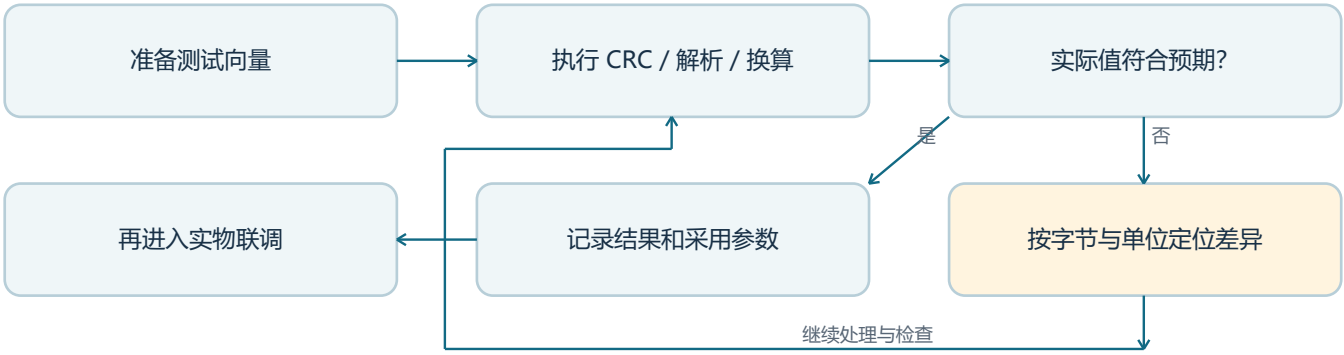
名称与类型	含义、用途与所属模块
测试记录	写实际结果、参数、失败和修改原因；不要用“预计正确”替代。
build / compile_commands.json	生成结果，可以重建；不能把它当源代码的唯一位置。

本步验收：通过后再继续

在新目录从无 build 状态可以重新配置和编译；演示覆盖所有必做项；README 能让他人知道怎么接线、怎么使能、怎么停止。

19 附录 A：协议测试帧与算法自检

本步目的：给通信和反馈算法提供可以重复核对的数字例子。



CRC 覆盖前 7 字节时的测试帧

意义	8 字节 HEX（末字节为 CRC）
速度：限流原始值 1000、目标 200 rpm	AA BB 00 E8 03 C8 00 29
速度：限流原始值 1000、目标 -200 rpm	AA BB 00 E8 03 38 FF CE
速度：限流原始值 1000、目标 0 rpm	AA BB 00 E8 03 00 00 6C
位置：最大 120 rpm、目标 720°	AA BB 11 78 00 D0 02 63
预设速度动作 0x00000000	AA BB 22 00 00 00 00 A5
预设位置动作 0x11111111	AA BB 22 11 11 11 11 28
反馈：速度到达、200 rpm、位置 720°	BB AA AF C8 00 D0 02 D3

这些帧是按本文明确假设计算的测试向量，不是从上位机抓包得到的。第一条速度样例仅用于解码测试；电机上电调试必须遵守限流、使能和安全步骤，不能看到帧就直接让电机全程运行。

组帧测试清单

输入方式	应有结果
合法帧一次发 8 字节	解析 1 条命令。
合法帧分 3+5 字节	收到前 3 字节不输出；后 5 字节到齐后解析 1 条。
两条合法帧连续 16 字节	解析 2 条，顺序正确。
在合法帧前插入 01 02 AA 03	跳过噪声后找到正确帧。
翻转合法帧第 5 字节，CRC 不变	拒绝坏帧；后续正确帧仍能恢复。
超过一个 DMA 环的连续流	跨 128/256 边界仍无重复和缺漏。
只发错 CRC，持续超过主机超时	不能维持在线；进入安全态。

角度展开的确定性例子

相邻原始角度	展开后 delta	解释
100 → 120	+20	正常正向。
8190 → 2	+4	正向跨零。
2 → 8190	-4	反向跨零。
100 → 4196	恰好半圈，不应认定方向	半圈边界有歧义，应作为采样连续性异常处理。

步骤 11 已把恰好 ±4096 的歧义跳变返回为失败；控制任务应据此锁存位置失效。同时结合最大转速与反馈间隔做合理性检查。只靠一个跨零公式无法检测所有丢整圈情况。

时间与状态测试

项目	预期
速度动作 elapsed=0/1000/2000 ms	60/120/180 rpm。
elapsed=3000/5000/7000/8000 ms	240/300/150/0 rpm。
elapsed=9000 ms	返回完成，目标保持 0。
PID limit=0	返回 0，并清动态状态。
持续正误差但输出达上限	积分不能继续向正方向无限增大。
系统 tick 接近 UINT32_MAX 后回绕	uint32_t(now-last) 仍能测短间隔；不要使用带符号比较。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
本附录的测试函数	桌面自检与板上调试都可使用同一组向量；本教程只验证算法预期，不代替 DMA、CAN 和真实电机测试。

变量说明：名称含义与保存内容

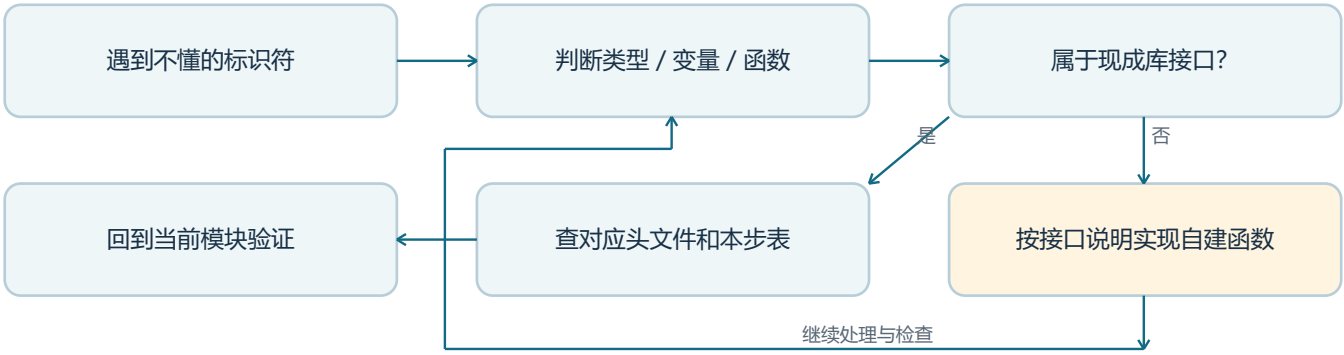
名称与类型	含义、用途与所属模块
test_start_tick / elapsed_ms	一个是动作开始时间，一个是时间差；重复心跳只更新连接时间，不更新动作开始时间。

本步验收：通过后再继续

算法数字例子全部通过；再用上位机抓包核对 CRC 覆盖范围和 A 的单位。两者不一致时先修正协议约定，再调 PID。

20 附录 B：C 语法、接口边界与资料索引

本步目的：把容易忘记的语法和函数来源放在同一处，方便边写边查。



常见 C 语法速查

写法	含义与本工程用途
uint8_t / int16_t / uint32_t	分别是无符号 8 位、有符号 16 位、无符号 32 位整数。字节流用 uint8_t，协议速度用 int16_t，时间戳用 uint32_t。
100U / 100.0f	U 表示无符号整数，f 表示 float；速度/角度计算用 float，并不是名字里带 f 的变量才是浮点数。
0xAA	十六进制数值 170；它是一个字节数值，不是字符串 "AA"。
1U << 2	无符号 1 左移两位，得到第 2 位的掩码 4；位编号从 0 开始。
flags & MASK / a | b	& 按位检查哪些位为 1；| 合并位。不是逻辑 && / ||。
&object / pointer->field	& 取地址；-> 通过指针访问结构体字段。函数接收指针可以填写调用者准备的对象。
const uint8_t *p	函数不通过这个指针修改所指向的字节；指针仍可用于读取数组。
static	文件作用域限制可见性；函数内 static 保留跨调用生命周期。不同作用域的用途要区分。
extern	声明某个变量在其他 .c 中已经定义；不是再分配一份队列。
volatile	提示访问不能随意优化掉，常用于硬件/中断标志；不提供互斥，也不保证结构体多字段一致。
sizeof(type) / sizeof(array)	取占用字节数；函数参数里的数组已退化为指针时，sizeof 不再是原数组总长度。
#ifndef / #define / #endif	头文件防重复展开；保护宏名不强制等于文件名，但必须足够独特。

.h 与 .c 怎么配合

.h 是公开接口：类型、宏、extern 变量声明和函数声明；.c 是具体实现和私有状态。一个 .c 可以包含多个 .h，因为它需要使用多组工具的声明，不意味着所有函数都被重复定义。.c 与 .h 同名只是便于管理，真正的联系是声明与实现一致、使用者包含正确头文件、实现文件参与编译。

头文件布局示例：必须先补入步骤 11 的完整 MotorFeedback typedef 才可编译；.c 包含该头文件，避免再定义一份相同类型。

```
/* motor_feedback.h */
#ifndef MOTOR_FEEDBACK_H
#define MOTOR_FEEDBACK_H
```

```
#include <stdbool.h>
#include <stdint.h>
#include "app_types.h"
/* Put MotorFeedback typedef from step 11 here. */
bool MotorFeedback_Update(MotorFeedback *m, const CanFrame *f,
                          float ratio, float direction);
#endif
```

"motor_feedback.h" 通常用于项目头文件；<stdint.h> 用于标准库。双引号会优先尝试当前文件相关目录再找配置的 include 路径；尖括号按编译器的系统/include 搜索规则查找。它们不会决定函数是库函数还是自己写的，函数来源要看实际声明与实现。

一组公共配置的起点

M3508+C620 示例；若改 M2006+C610，同步改比值为 36.0f、硬上限为 4000。MOTOR_DIR 以实测为准。集中定义后删除各 .c 中重复同名宏。

```
#ifndef APP_CONFIG_H
#define APP_CONFIG_H
#define APP_READY            (1U << 0)
#define TX_DONE              (1U << 1)
#define TX_ERROR             (1U << 2)
#define COMM_FAULT           (1U << 3)
#define CAN_FAULT            (1U << 4)
#define HOST_TIMEOUT_TICKS   300U
#define CAN_TIMEOUT_TICKS    50U
#define CONTROL_PERIOD_TICKS 1U
#define REPORT_PERIOD_TICKS  20U
#define MOTOR_DIR            1
#define MOTOR_GEAR_RATIO     (3591.0f / 187.0f)
#define MOTOR_HARD_LIMIT     8000
#endif
```

这些 tick 参数假设 1000 Hz；若修改系统 tick，必须统一毫秒换算，不能只改某一个超时值。CAN_TIMEOUT_TICKS 是通信离线阈值，不能代替角度展开要求的更短连续性阈值。

资料来源与需要现场确认的事项

依据	用于核实什么
WTR最终考核.md / .pdf	任务、协议、引脚原理图、8 MHz 晶振、提交要求。
附件/crc.hpp	CRC8 参数和高位优先算法，本文改写成 C。
C610 使用说明，PDF 第 8～9 页	1 Mbps、标准数据帧、ID、编码范围与电流缩放。
M2006 使用说明，PDF 第 2/9 页	减速比 36:1。
C620 使用说明 V1.01，PDF 第 14 / 32 页	电流指令 ±16384 对应 ±20 A；反馈字节布局。
M3508 使用说明 V1.0，PDF 第 13 页	精确减速比 3591/187。
CoreV2.1.pdf	仅用于识别板版本冲突；未将其 FDCAN 等资源当成本次 F407 配置。

资料中的 Engine/BSP/DJI 链接是可参考实现，本文没有把未读取的网络代码当成已验证依据。上位机可执行文件没有被运行或反编译；其实际波特率、CRC 覆盖范围、A 单位、动作重复触发行为必须通过界面和抓包确认。

库接口的官方参考

ST：F407 引脚数据

https://github.com/STMicroelectronics/STM32_open_pin_data

ST：UART HAL 驱动源码

https://github.com/STMicroelectronics/stm32f4xx-hal-driver/blob/master/Src/stm32f4xx_hal_uart.c

ST：循环 DMA + IDLE 接收示例

https://github.com/STMicroelectronics/STM32CubeF4/tree/master/Projects/STM32446E-Nucleo/Examples/UART/UART_ReceptionToIdle_CircularDMA

Arm：CMSIS-RTOS2 消息队列

https://arm-software.github.io/CMSIS_6/main/RTOS2/group__CMSIS__RTOS__Message.html

Arm：CMSIS-RTOS2 线程标志

https://arm-software.github.io/CMSIS_6/main/RTOS2/group__CMSIS__RTOS__ThreadFlagsMgmt.html

接口查阅日期：2026-10-20。工程应以实际安装的 STM32CubeF4 和 CMSIS 头文件为最终版本依据。官方 F446 示例用于核实 HAL 接收事件语义，不是把 F446 板的引脚和时钟复制到 F407。

函数说明：谁提供，谁调用，为什么需要

名称 / 来源	目的、参数与结果
名称前缀 HAL_	通常来自 STM32 HAL；名字是回调的函数，其函数体需要用户按规则实现。
名称前缀 os	本文来自 CMSIS-RTOS2；底层可以是 FreeRTOS，不要混用 CMSIS v1 的旧函数签名。
Protocol_ / MotorCan_ / PID_ / Preset_ / Control_	本教程自建模块接口，名字本身不会让编译器自动生成实现。凡标为建议接口或伪代码，都需在对应步骤完成定义。

变量说明：名称含义与保存内容

名称与类型	含义、用途与所属模块
功能变量与调试变量	被决策逻辑读取的是功能变量；仅用于观察统计、删掉不影响状态流转的才可能是纯调试变量。

本步验收：通过后再继续

能够从任意一行关键代码追到：变量由谁保存，函数由谁提供，调用后得到什么，以及下一步如何验证。正式动手时，从步骤 02 ~ 04 建立并编译一个最小可运行工程。